

BITÁCORAS BASE DE DATOS II

2020-2021

2º curso – 2º cuadrimestre

Os seguintes apuntamentos foron recollidos en forma de bitácoras durante cada clase do curso polas seguintes persoas:

Adriana Aurora Rodríguez Oreiro, Adrián Vidal Lorenzo, Andrea Solla Alfonsín, Andrés Balea Domínguez, Belén Valle Cao, Diana Mascareñas Sande, Elena Segade Martínez, Hugo Vázquez Docampo, Inés Quintana Raña, Javier Beiro Piñón, Manuel Cid Domínguez, Martín Campos Zamora, Miguel Méndez Martínez, Nerea Freiría Alonso, Nicolás Vilela Pérez, Pablo Gil Pérez, Pablo Martínez González, Roberto de la Iglesia, Roque Fernández Iglesias, Teresa Gutiérrez Blanco

Kahoot!



Non son apuntamentos recollidos de forma 100% formal e pode haber erros nalgúns datos ou fallos de formato. A pesar disto, pensamos que poden ser de gran axuda para os vindeiros cursos :)



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Bases de Datos II

08-02-2021



Clase 1

Temas trabajados: Presentación de la materia

Índice

1. Conceptos.....	3
2. Evaluación	3
Fecha exámenes.....	3
3. Bibliografía	3
4. Correos de los profesores	3
5. Calendario	4

1. Conceptos

ACID → Atomicidad, Consistencia, Aislamiento y Durabilidad

LAMP → Linux, Apache, MySQL/MariaDB y Perl/PHP/Python. Hace referencia a un conjunto de tecnologías informáticas –software– que se emplean en el desarrollo web.

2. Evaluación

Media geométrica entre el examen y la continua

- Examen: 3 partes, cuestiones, aplicaciones (mínimo 5), sql (mínimo 5).
- Continua: rúbrica de evaluación del proyecto (aún no subida en el campus)
 - En los grupos de los proyectos aún está por decidir que sean aleatorios o no y el número de integrantes
- Evaluación de Competencias
 - **Examen:** CB4, CG4, CG8, TR3, FB4
 - **Evaluación Continua:** CG2, CG9, TR1, TR2, TR3, RI1, RI5, RI12, RI13, RI14, TI5

Al principio de cada clase se tratarán conceptos que **no serán** (en la mayoría de los casos) **material de examen**. Serán cuestiones sobre temas de interés.

Asistencia a prácticas no obligatoria: algunas específicas tendrán asistencia obligatoria para modificaciones de programa o similares.

Puntos adicionales para compensar o subir nota por participación, no especificados aún.

Fecha exámenes

Primera oportunidad: lunes 7 de junio 9:15.

Segunda oportunidad: jueves 8 de julio 16:00.

3. Bibliografía

[Enlace a la bibliografía de la materia](#)

4. Correos de los profesores

joaquin.trinanes@usc.es

antonio.mosquera@usc.es

5. Calendario

BASES DE DATOS 2		Docencia Expositiva	Docencia Interactiva	Entregas
S1	lunes, 1 de febrero de 2021	T0. Presentación Materia		
	miércoles, 3 de febrero de 2021			
	jueves, 4 de febrero de 2021			
	viernes, 5 de febrero de 2021			
S2	lunes, 8 de febrero de 2021	T1. DyD Aplicaciones de BD	Java/JDBC (Sesión 1)	
	miércoles, 10 de febrero de 2021			
	jueves, 11 de febrero de 2021			
	viernes, 12 de febrero de 2021			
S3	lunes, 15 de febrero de 2021			
	miércoles, 17 de febrero de 2021		Java/JDBC (Sesión 2)	
	jueves, 18 de febrero de 2021			
	viernes, 19 de febrero de 2021			
S4	lunes, 22 de febrero de 2021	T2. Gestión de Transacciones	Java/JDBC (Sesión 3)	
	miércoles, 24 de febrero de 2021			
	jueves, 25 de febrero de 2021			
	viernes, 26 de febrero de 2021			
S5	lunes, 1 de marzo de 2021	T2. Gestión de Transacciones	Java/JDBC (Sesión 4)	
	miércoles, 3 de marzo de 2021			
	jueves, 4 de marzo de 2021			
	viernes, 5 de marzo de 2021			
S6	lunes, 8 de marzo de 2021	T2. Gestión de Transacciones	EVALUACIÓN PRÁCTICA Java/JDBC	PRÁCTICA Java/JDBC (INDIVIDUAL)
	miércoles, 10 de marzo de 2021			
	jueves, 11 de marzo de 2021			
	viernes, 12 de marzo de 2021			
S7	lunes, 15 de marzo de 2021	T3. Structured Query Language	PROYECTO (Sesión 1)	
	miércoles, 17 de marzo de 2021			
	jueves, 18 de marzo de 2021			
	viernes, 19 de marzo de 2021			
S8	lunes, 22 de marzo de 2021	T4. Funcionamiento Interno de los SGBD	PROYECTO (Sesión 2)	
	miércoles, 24 de marzo de 2021			
	jueves, 25 de marzo de 2021			
	viernes, 26 de marzo de 2021			
S9	lunes, 29 de marzo de 2021			
	miércoles, 31 de marzo de 2021			
	jueves, 1 de abril de 2021			
	viernes, 2 de abril de 2021			
S10	lunes, 5 de abril de 2021	T4. Funcionamiento Interno de los SGBD	PROYECTO (Sesión 2)	
	miércoles, 7 de abril de 2021		PROYECTO (Sesión 3)	
	jueves, 8 de abril de 2021			
	viernes, 9 de abril de 2021			
	lunes, 12 de abril de 2021	T4. Funcionamiento Interno de los SGBD		
S11	miércoles, 14 de abril de 2021		PROYECTO (Sesión 4)	PROYECTO GRUPO
	jueves, 15 de abril de 2021			
	viernes, 16 de abril de 2021			
S12	lunes, 19 de abril de 2021	T5. Arquitecturas de los Sistemas de BD	PRESENTACIÓN PROYECTO	
	miércoles, 21 de abril de 2021			
	jueves, 22 de abril de 2021			
S13	viernes, 23 de abril de 2021		APORTACIÓN INDIVIDUAL A PROYECTO GRUPO	
	lunes, 26 de abril de 2021	T6. Gestión Avanzada de la Información		
	miércoles, 28 de abril de 2021			
S14	jueves, 29 de abril de 2021			
	viernes, 30 de abril de 2021			
	lunes, 3 de mayo de 2021	T7. Temas Avanzados		
S15	miércoles, 5 de mayo de 2021			
	jueves, 6 de mayo de 2021			
	viernes, 7 de mayo de 2021			
	lunes, 10 de mayo de 2021	NO CLASE		
S15	miércoles, 12 de mayo de 2021			
	jueves, 13 de mayo de 2021			
	viernes, 14 de mayo de 2021			



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Bases de Datos II

22-02-2021



Clase 2

Temas trabajados: Tema 1

Índice

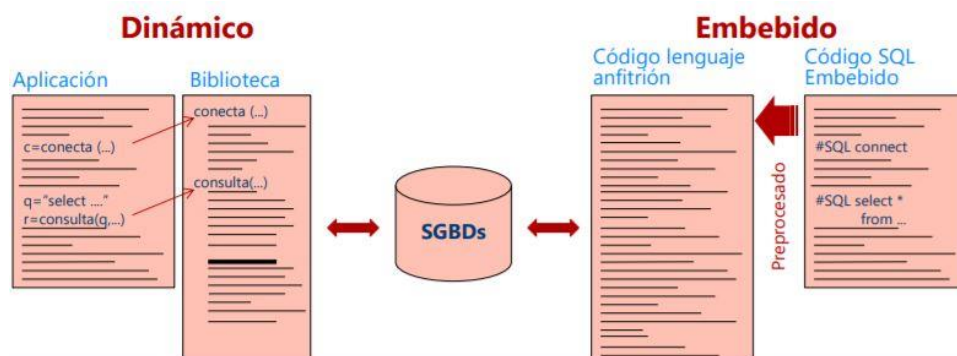
1. Introducción	3
1.1 Mayor desafío	3
2. JDBC.....	3
2.1 Prepared Statements	4
2.1.1 Inyección SQL	4
2.1.2 Sentencias que se pueden invocar	5
2.1.3 Metadatos	5
2.1.4 Transacciones.....	5
2.1.5 Objetos grandes	5
3. ODBC	5
3.1 Estructura de un programa ODBC.....	5
3.2 Gestión de las transacciones.....	5
4. SQL embebido	6
4.1 Estructura	6
4.2 SQLJ	6
5. Definiciones	7
1. Common Table Expression.....	7
2. Sistema Turing Complete	7
3. API o interfaz de programación de aplicaciones.....	7
4. ResultSet	7
5. Prepared statements	7

1. Introducción

No todas las consultas se podían realizar en SQL al tener menor potencia expresiva, pero, con el uso de las *Common Table Expressions* ¹ ya es *Turing Complete* ², es decir, ya se puede realizar cualquier tipo de consulta, incluidas las recursivas. Esto puede ser muy complejo, y, por tanto, es más fácil llevarlo a cabo en un entorno que tengamos más controlado (Python, Java, C, etc.). Por ello, interesa incorporar esas estructuras de SQL en nuestro entorno de trabajo y realizar operaciones como el cálculo de agregaciones o computaciones sobre los datos extraídos de la base de datos. Así, podremos sacarles partido o visualizarlos de una determinada forma.

Existen dos formas de realizar estas consultas: SQL dinámico y SQL embebido.

- Con **SQL dinámico** podemos generar SQL en tiempo de ejecución ([run time](#)), para ello utilizamos estándares como JDBC u ODBC que permiten incluir sentencias SQL dentro del programa; que después, una librería es capaz de traducir para utilizar el API ³ que nos ofrecen estos drivers y poder interaccionar con la base de datos.
- **SQL embebido** permite preprocesar esas consultas, es decir, si tenemos un programa en C nos permite embeber el código SQL de tal forma que un procesador lo modifica y lo pasa directamente a funciones del lenguaje anfitrión.



En SQL embebido primero se traduce todo el código SQL pasándolo al lenguaje utilizado (que interacciona con el SGBDs). Esto permite detectar errores de sentencias SQL en tiempo de compilación mientras que el SQL dinámico no.

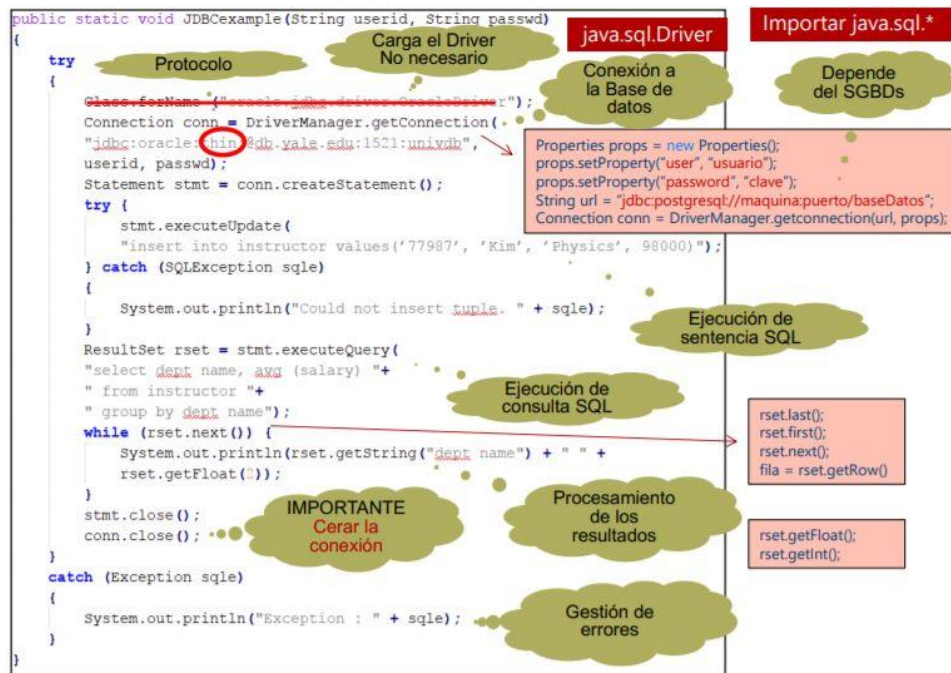
1.1 Mayor desafío

El mayor desafío es la forma de trabajar con los datos. En SQL trabajamos con *result sets* ⁴ y una serie de operaciones con relaciones (tablas). Al utilizar esas operaciones dentro del lenguaje anfitrión, pueden surgir dificultades en relación a los tipos de datos ya que los utilizados en el entorno de programación pueden ser muy diferentes a los que utilizamos en la base de datos.

2. JDBC

Ofrece una API que nos permite conectarnos a una base de datos e interaccionar con ella.

Funcionamiento del driver de JDBC: una librería nos ofrece los protocolos que nos permiten llamar a la base de datos de una forma estándar sacándole el máximo partido, utilizando para ello la interfaz *java.sql.Driver*. El SGBDs debe proporcionar una clase que implemente dicha interfaz. Es necesario introducir el *.jar* proporcionado por el fabricante dentro del directorio en el que se almacenan las clases. Cabe destacar que podemos indicar el protocolo a utilizar ya que un fabricante puede tener varios.



- Si incluimos el `.jar` no es necesario incluir ese comando (por eso aparece tachado).
- Sobre un `result set` podemos ir al primer o último elemento, o posicionarnos sobre uno concreto.
- Siempre hay que gestionar lo que nos devuelve la base de datos con un bloque try-catch, porque si pasa algo y no lo gestionamos la interfaz no va a responder a lo que pedimos. Ejemplo: si surge un problema en la conexión y no está contemplado con un try-catch la interfaz no va a responder de forma esperada.

2.1 Prepared Statements

En lo relativo a los *prepared statements* ⁵, si se modifica uno de los parámetros los otros se mantienen. Son más eficientes que los Statements.

Importante: Existe una serie de caracteres especiales que su uso puede producir una excepción. Algunos de estos son: `'`, `\`, `\n`, `\t`, `"`, `tab`, `retorno de carro`...

2.1.1 Inyección SQL

Se puede evitar con las sentencias preparadas, ya que primero se carga la consulta y luego se introducen las variables. Ejemplo:

1. `"select * from instructor where name = " + name + " and passwd="+passwd+""`
2. El usuario teclea en el campo passwd el valor: `"X" or 'Y' = 'Y'`
 - Si se usa `passwd = "+passwd+"`, para entrar se comprueba si `passwd = X` o si `Y = Y` (por lo que será siempre cierto) -> se obtendría acceso si se cumpliese cualquiera de las dos partes del or.
 - Si se usa un prepared statement comprueba si passwd es igual a la cadena `"X"` or `"Y"='Y'` (será falso).

Ejemplo práctico muy claro:

Si te dicen "Ve a buscar al aeropuerto a", sabes que tienes que ir a recoger a alguien al aeropuerto. Si a continuación te dicen el nombre: "Juan y mata a Fulanito" vas a buscar a una persona con ese nombre, que no va a existir y por tanto no la vas a encontrar (prepared

statements). Si, por el contrario, te dicen “Ve a buscar al aeropuerto a Juan y mata a Fulanito” interpretas que tienes que hacer esas 2 cosas (Strings concatenados).

Por lo tanto, si quieres proteger a Fulanito, debes usar *prepared statements*, que son la protección de JDBC.

2.1.2 Sentencias que se pueden invocar

- **Interfaz *CallableStatement*** para procedimientos y funciones:
 - `CallableStatement c1 = conn.prepareCall ("? = call some function(?) ");`
 - `CallableStatement c2 = conn.prepareCall ("? = call some procedure(?,?) ");`
- **Registro de parámetros:** `registerOutParameter`. (Obtención de valores: `get...`)

2.1.3 Metadatos

Permite conocer el nombre de las relaciones (tablas), el nombre de las columnas (atributos)...

```
DatabaseMetaData dbmd = conn.getMetaData();
```

```
ResultSet rs = dbmd.getColumns(null, "univdb", "department", "%");
```

2.1.4 Transacciones

Por defecto cada sentencia se ejecuta en una transacción distinta (método **`setAutoCommit(true)`**). En las transacciones atómicas no es necesario utilizar `commit()` o `rollback()`.

2.1.5 Objetos grandes

Normalmente están almacenados o se quieren almacenar en un fichero. Usando el método `setBlob` enlazamos el objeto a un `Stream` para poder trabajar con él. Además, los métodos `getBlob` y `getClob` permiten obtener la localización de dichos objetos. Por último, para obtener los objetos usamos `getBytes` o `getSubString`.

3. ODBC

Es similar y anterior a JDBC y, al igual que él, nos ofrece una API para interaccionar con la base de datos.

3.1 Estructura de un programa ODBC

1. Definir el entorno de trabajo
2. Conectarse a la base de datos (muy parecido a JDBC, pero con algunas variaciones)
3. Reservamos la memoria para la consulta con `SQLDirect`.
4. Asociamos lo que vamos a recibir de la consulta con las variables que tenemos definidas en nuestro programa.
5. Hay que definir el tamaño máximo del resultado que vamos a obtener.

3.2 Gestión de las transacciones

Se pueden ejecutar transacciones de una sola operación o de varias. Ofrece la posibilidad de hacer `AutoCommit` pero, si no está activado, hay que introducir `commit()` o `rollback()` de forma explícita.

El estándar de SQL define *Call Level Interface* (CLI). En lo relativo a ODBC, el CLI puede interpretarse como una API ya que, debido a su popularidad durante el periodo de estandarización de SQL, se incluyó en él.

También existe el API de ADO.NET, creado para lenguajes Visual Basic y C#. Se apoya en una extensión de ODBC y se caracteriza porque se puede usar con algunas fuentes no relacionales.

4. SQL embebido

El programa ha de ser preprocesado antes de poder ser compilado con el compilador del lenguaje anfitrión. Para ello, identificamos una serie de etiquetas en el programa que le indican al procesador las partes a traducir para poder compilar e interaccionar con la base de datos. Como mencionamos anteriormente, los errores de sintaxis se detectan en tiempo de compilación a diferencia de SQL dinámico.

4.1 Estructura

1. Cabeceras (las estructuras y variables necesarias se introducen con *SQL INCLUDE SQLCA*).
2. Nos conectamos al servidor con un usuario y una contraseña.
3. Declaramos unas variables de intercambio entendidas por el servidor y el lenguaje anfitrión que permiten meter las variables de la base de datos en nuestro entorno y viceversa.
4. Ejecutamos la sentencia SQL.
5. Accedemos a los valores de las variables. Con ':' referenciamos el valor que tiene cada una de esas variables y le decimos al programa que tiene que usar esos valores.
6. Declaramos un cursor (puntero dinámico a un *ResultSet*) para, por ejemplo, realizar una consulta.
7. Abrimos ese cursor y, para cada uno de los elementos devueltos por este, los metemos dentro de las variables de lenguaje anfitrión.
 - ♦ **SQLSTATE**: permite varias cosas, como indicar que no hay más tuplas en el *ResultSet* (Gestión de errores). Son estándares en las bases de datos.
 - ♦ **SQLCODE**: tiene un rango de valores pequeños e indica el éxito o fracaso de la ejecución. No hay estándar y los desarrolladores definen sus propios valores por lo que es recomendable usar SQLSTATE.
8. Ejecutamos la consulta.
9. Procesamos el resultado.
10. Liberamos el cursor.

Nota: un cursor, aparte de recibir datos, permite modificar y actualizar tuplas en la base de datos (for update). A la vez que realizas una consulta a la base de datos puedes realizar una actualización sobre los campos correspondientes a esa tupla lo cual es una de las grandes ventajas de utilizar SQL desde una aplicación. Si algo va mal, yo confirmo o hago un rollback a la situación anterior.

4.2 SQLJ

Es SQL embebido en entornos java. Es casi como JDBC, de hecho, existen programas que mezclan ambos y las estructuras son compatibles. Es un estándar desarrollado por diversos fabricantes que constituye la parte 10 del estándar *SQL Object Language Bindings* (SQL/OLB). Existen varios operadores como la Interfaz iterator (para más de una tupla) o #.

En definitiva, en lugar de ser código embebido en c, c++, fortran... es código embebido en java (combinación java-SQL).

5. Definiciones

1. Common Table Expression: Conjunto de resultados con nombre temporal al que puede hacer referencia dentro de una instrucción SELECT, INSERT, UPDATE o DELETE.
2. Sistema Turing Complete: Sistema en el que se puede escribir un programa que encontrará una respuesta (aunque sin garantías con respecto al tiempo de ejecución o la memoria). Si alguien dice "mi nueva cosa es Turing Complete", eso significa en principio podría usarse para resolver cualquier problema de cálculo.
3. API o interfaz de programación de aplicaciones: Conjunto de subrutinas, funciones y procedimientos que ofrece cierta biblioteca para ser utilizado por otro *software* como una capa de abstracción. Ejemplos:
 - ◆ Usar Google Maps en una aplicación de uso compartido de transporte.
 - ◆ Crear bots de chat en un servicio de mensajería.
 - ◆ Integrar vídeos de YouTube en una página web.
4. ResultSet: contiene los resultados de una consulta SQL, por lo que es un conjunto de filas obtenidas desde una base de datos, así como Metadatos sobre la consulta y los tamaños de cada columna.
5. Prepared statements: Strings con '?' que luego se completan con los valores requeridos



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Bases de Datos II

07-03-2021

Clase 03

Temas trabajados:

2. Gestión de Transacciones (Parte 1)

Índice

1. Kahoot	3
-----------------	---

1. Kahoot

1. **Se denomina transacción a un conjunto de operaciones ...** que forman una unidad lógica de trabajo | consecutivas relacionadas por temática | a realizar en una Base de Datos desde un lenguaje de programación | consecutivas del tipo SELECT ... FROM ... WHERE.
Lo importante es que tengan una operación lógica.
2. **Una transacción puede realizarse de forma NO completa en el siguiente caso.** Nunca | Siempre | Sólo cuando está aislada | Siempre que no sea concurrente
El conjunto de operaciones que forma una transacción es indivisible.
3. **¿Es posible la ejecución a la vez de varias transacciones?** Sí, siempre que no se introducen inconsistencias en los datos | Sí, siempre | Sí, siempre que afecten a datos diferentes | No, nunca
Lo que no se puede hacer es querer hacer varias cosas a la vez y que esas cosas se entremezclen sin control y que el resultado global sea diferente al que tendríamos si hubiésemos hecho varias.
4. **O bien todas las operaciones de la transacción se realizan adecuadamente o no lo hace ninguna de ellas.** Atomicidad (Atomicity) | Consistencia (Consistency) | Aislamiento (Insolation) | No es una definición de una propiedad ACID
Cuando se tiene una unidad lógica, o bien en la BD se ejecutan todas las operaciones que estén definidas en la transacción, o, si por algún motivo pasa algo (fallo del sistema, fallo de la lógica de la transacción, etc.), hay que deshacer todo.
5. **La ejecución aislada de una transacción, bien codificada, no tiene porqué conservar la consistencia de la base de datos.** No es una definición de una propiedad ACID | Atomicidad (Atomicity) | Durabilidad (Durability) | Consistencia (Consistency)
La definición de consistencia es justamente lo contrario.
6. **Cuando se ejecutan transacciones de forma concurrente, éstas comparten los datos comunes ejecutándose todas o ninguna.** No es una definición de una propiedad ACID | Atomicidad (Atomicity) | Aislamiento (Insolation) | Durabilidad (Durability)
No es la definición del concepto de aislamiento. La posibilidad de hacer transacciones concurrentes que pueden compartir datos comunes es verdadera, pero la condición no es que se ejecuten todas o ninguna, porque existe la posibilidad de que algunas se ejecuten y otras no.
7. **Tras el éxito de una transacción, los cambios realizados permanecen, incluso aunque se produzcan fallos en el sistema.** Durabilidad (Durability) | Consistencia (Consistency) | Aislamiento (Insolation) | No es una definición de una propiedad ACID
Es la definición de durabilidad.
8. **¿Qué propiedad ACID es responsabilidad del programador que codifica la transacción?** Consistencia (Consistency) | Atomicidad (Atomicity) | Aislamiento (Insolation) | Durabilidad (Durability)
Las otras 3 no son responsabilidad del programador, es responsabilidad del gestor de la BD.
9. **Modificación en la BD realizada por transacción vs. anotación en registro de modificación que realizará la transacción.** Primero se anota en el registro de modificación y luego se ejecuta | Primero se ejecuta la modificación y luego se anota en el registro | Se hacen simultáneamente para garantizar la atomicidad y la durabilidad | El orden no es importante
10. **Estado tras descubrir que no puede continuar la ejecución normal (errores de hardware o lógicos).** Transacción Fallida | Transacción Activa | No es un estado de una transacción | Transacción Comprometida

11. **Estado en el que permanece la transacción durante su ejecución.** *Transacción Activa* | *Transacción Fallida* | *No es un estado de una transacción* | *Transacción Comprometida*
Cuando la transacción comienza y mientras se está ejecutando.
12. **Estado tras descubrir que una transacción ha resultado fallida sólo mientras se consulta el registro de operaciones.** *No es un estado de una transacción* | *Transacción Activa* | *Transacción Abortada* | *Transacción Parcialmente Abortada*
Para que se considere abortada tiene que haberse reestablecido la BD al estado anterior a la transacción. Una transacción que da problemas se pone como fallida y luego se hacen las operaciones de garantizar la atomicidad y, por lo tanto, de deshacer al estado inicial. Cuando se deshacen se pasa a transacción abortada.
13. **¿Existe un estado de una transacción denominado “Parcialmente comprometida”?** *Sí, después de terminar pero antes de que la información esté grabada* | *Sí, tras la ejecución de cada sentencia, salvo la última* | *Sí, después de la primera sentencia hasta que se ejecuta la última* | *No; Del estado “Activa” se pasa al estado “Comprometida”*
Si yo las operaciones las hago en el almacenamiento no volátil, el sistema es lentísimo. Entonces, lo que se hace es coger bloques de información y llevarla a la memoria volátil. En esta memoria, si se cae el sistema, se pierde la información, por lo que luego hay que conseguir que esos datos vayan a una memoria no volátil y de ahí el estado “Parcialmente comprometida”.
14. **¿Se pueden deshacer los cambios realizados por una transacción comprometida?** *Sólo mediante una transacción compensadora realizada por el usuario* | *Retrocediendo la transacción a parcialmente comprometida y abordándola* | *El sistema hace transacciones compensadoras cuando detecta la necesidad* | *No, nunca; Va en contra de la durabilidad*
Es imposible que el sistema detecte la necesidad de transacciones compensadoras. No se puede decir nunca, porque el programador puede hacerlo. No hay retrocesos en los estados de las transacciones.
15. **¿Cuál NO es una ventaja de la ejecución concurrente de transacciones?** *Reducción de fallos del sistema* | *Mejora del rendimiento del sistema* | *Mejora del uso de recursos* | *Reducción del tiempo de espera*
De las 3 ventajas, la más importante es la reducción del tiempo de espera, porque hay transacciones que tardan poco y otras mucho, entonces si las que tardan poco van después de las que tardan mucho, tendrán que esperar. Si esas que tardan poco no afectan a las tablas de las que tardan mucho, sería interesante que se puedan hacer de manera simultánea.
La concurrencia seguramente sea lo que más complica los sistemas.
16. **¿Qué es una planificación?** *El orden de ejecución de las instrucciones de un conjunto de transacciones* | *El orden de ejecución de las transacciones de un conjunto de transacciones* | *El estudio de las concurrencias, a nivel de datos, de las transacciones* | *El estudio de las concurrencias, a nivel transacciones, de los datos*
Nunca se puede cambiar el orden de las instrucciones de una transacción.
17. **Todas las planificaciones posibles generan un estado consistente en la base de datos.** *Falso*
El concepto de planificación solo hace referencia al orden de las instrucciones. Todas las combinaciones posibles son válidas, pero no todas van a dar estados consistentes en la BD.
18. **Todas las planificaciones posibles, que generan estados consistentes, llegan al mismo estado final.** *Falso*
Ej: operación aritmética y el producto. Si yo hago la operación aritmética antes que el producto, se hace la cuenta sobre el resultado de la aritmética; pero si hago el producto y luego la aritmética, el producto se hace sobre el valor de origen de la aritmética, al cual se le suma después algo, que no lleva la contribución del producto.

19. **Planificación secuencial: secuencia de instrucciones de varias transacciones ...** en la que todas las instrucciones de cada transacción están juntas | en la que las instrucciones de lectura de cada transacción están juntas | en la que las instrucciones de escritura de cada transacción están juntas | en la que las instrucciones de cada transacción están divididas en grupos

Si todas las instrucciones de cada transacción están juntas, es que estoy ejecutando una transacción detrás de otra (es la única forma posible), y ahí no tengo ningún problema de aislamiento de transacciones.

20. **Una planificación que asegure que la ejecución concurrente tiene el mismo efecto que sin concurrencia se denomina ...** planificación serializable | planificación secuencial | planificación concurrente | planificación aislada

Transacciones que aseguran la ejecución de forma concurrente pero que el efecto que tienen en la BD es como si la planificación fuera secuencial (una transacción tras otra).



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Bases de Dato II

15-03-2021

Clase 4

Temas trabajados:

Kahoot: Gestión de Transacciones (Parte 2)

Índice

1. Kahoot.....	3
----------------	---

1. Kahoot

1. Cuando una planificación se puede transformar en otra mediante pasos no conflictivos se dice que ambas son ...

Equivalentes | Isomorfas | Inconcurrentes | Secuenciales

Esto se debe a que se corresponde a la definición de planificaciones equivalentes en cuanto a conflictos, donde si una planificación S se puede transformar en otra S' mediante una serie de intercambios de instrucciones no conflictivas, se dice que S y S' son equivalentes en cuanto a conflictos

2. Una planificación secuencial es siempre secuenciable.

Verdadero

Se dice que una planificación S es secuenciable en cuanto a conflictos si es equivalente en cuanto a conflictos a una planificación secuencial. Consecuentemente, una planificación secuencial ya es secuenciable en cuanto a conflictos con respecto a sí misma.

3. Instrucciones de lectura/escritura en dos transacciones sobre datos diferentes, ¿generan conflictos al intercambiarlas?

No, nunca | Si, siempre | Si, si las dos son de lectura | Si, si al menos una es de escritura

Teniendo en cuenta la definición de conflicto: Se dice que I y J están en conflicto si existen operaciones de diferentes transacciones sobre el mismo elemento de datos, y al menos una de esas instrucciones es una operación de escribir. Consecuentemente, al tratarse de elementos diferentes da igual que se haya instrucciones de lectura/escritura en dos transacciones, ya que no van a producir nunca ningún tipo de conflicto

4. ¿Cuál de las siguientes parejas de instrucciones NO genera conflictos al intercambiarlas?

Leer(A) en T1 con leer(A) en T2 | Leer(A) en T1 con escribir(A) en T2 | Escribir(A) en T1 con leer(A) en T2 | Escribir(A) en T1 con escribir(A) en T2

Esto se debe a que, como se ha comentado previamente, solo se produce un conflicto si existen operaciones de diferentes transacciones sobre el mismo elemento de datos, y al menos una de ellas es una operación de escritura. Consecuentemente, la opción correcta será en la que solo hay instrucciones de lectura.

5. ¿La planificación mostrada es secuenciable?

Si

Atendiendo a la definición de planificación secuenciable, podemos ver que esta planificación no presenta ningún posible conflicto, ya que las operaciones de lectura y escritura de los elementos se realizan en una transacción

T1	T2
leer(A)	leer(C)
leer(B)	escribir(C)
escribir(A)	leer(D)
escribir(B)	escribir(D)

cada una, A y B en T1 y C y D en T2, por lo que podría ser secuenciable al poder realizar intercambios sin generar conflictos

6. ¿La planificación mostrada es secuenciable?

Si

Esto se debe a que, a pesar de que se realizan lecturas y escrituras de los mismos elementos en transacciones distintas, esta planificación se podría sustituir por otra donde se ejecutan todas las de T1 y después todas las de T2, de manera que serían equivalentes en cuanto a conflictos al realizarse las operaciones sobre A y B antes en T1 que en T2, y además, al ser T1 secuencial, se llega a la conclusión que la planificación mostrada es secuencial en cuanto a conflictos.

T1	T2
leer(A) escribir(A)	leer(A) escribir(A)
leer(B) escribir(B)	leer(B) escribir(B)

7. ¿La planificación mostrada es secuenciable?

No

Este caso sería igual al anterior, sin embargo, se puede ver que la operación de escribir(A) de T1 se hace después de las operaciones de lectura y escritura sobre A de T2. Esto produce conflictos ya que, si se intercambiasen estos bloques de instrucciones, generarían conflictos y no se obtendría el mismo resultado en la base de datos. Consecuentemente, no sería equivalente en cuanto a conflictos con una planificación secuencial.

T1	T2
leer(A) leer(B)	leer(A) escribir(A)
escribir(A) escribir(B)	leer(B) escribir(B)

8. ¿La planificación mostrada es secuenciable?

No

En este caso, se puede ver que las operaciones sobre A de T1 se realizan antes que las de T2, pero las operaciones sobre B se T1 se realizan después que T2. Consecuentemente, si se pretendiese transformar esta planificación en una secuencial, se producirían conflictos, por lo que no sería posible

T1	T2
leer(A) A = A - 50 escribir(A)	leer(B) B = B - 10 escribir(B)
leer(B) B = B + 50 escribir(B)	leer(A) A = A + 10 escribir(A)

9. ¿La planificación mostrada genera el mismo resultado que la planificación secuencial T1:T2 (primero T1: después T2)?

Si

A pesar de que en la pregunta anterior se ha dicho que no sería una planificación secuencial, fijándose en el resultado de la base de datos, este sería el mismo, ya que independientemente en el orden en el que se realicen las operaciones, como la suma y la resta son conmutativas, el resultado no cambiaría.

T1	T2
leer(A) A = A - 50 escribir(A)	leer(B) B = B - 10 escribir(B)
leer(B) B = B + 50 escribir(B)	leer(A) A = A + 10 escribir(A)

10. Para determinar la secuencialidad en cuanto a conflictos de una planificación se puede usar...

Un grafo de precedencia | Un árbol de precedencia | Un registro de precedencia | Una relación de precedencia

Para determinar la secuencialidad de una planificación se construye un grafo de precedencia y se comprueba que ese grafo no tenga ciclos. Si tiene, esa planificación no es secuenciable en cuanto a conflictos

11. Las aristas de un grafo de precedencia representan transacciones...

Con instrucciones no intercambiables por conflictos de lectura/escritura | prioritarias en el acceso a los datos | cuyo intercambio generaría planificaciones no secuenciables | que realizan operaciones con los mismos datos

En un grafo de precedencia, los vértices son el conjunto de transacciones de la planificación, y las aristas representan operaciones sobre los mismos datos que realizan las transacciones que las unen.

12. Una planificación recuperable es aquella en la que, si T2 lee datos que ha escrito T1,...

La transacción T1 se compromete antes que la transacción T2 | La transacción T2 se compromete antes que la transacción T1 | La transacción T1 no se compromete | La transacción T2 no se compromete

Por definición, una planificación recuperable es aquella que para todo par de transacciones T_i y T_j tales que T_i lee elementos de datos que ha escrito previamente T_j , la operación comprometer de T_i aparece antes que la de T_j . De esta manera, si la transacción T_i falla después que T_j haya leído datos, al no haber terminado T_j esta se puede recuperar a su estado antes de leer los datos de T_i .

13. ¿La planificación mostrada es recuperable?

Si

Por la definición dada anteriormente, es recuperable ya que T2 lee el dato A escrito por T1, y T1 se compromete antes que T2.

T1	T2
escribir(A) commit	leer(A) commit

14. ¿La planificación mostrada es recuperable?

No

De la misma manera que en el caso anterior. T2 lee el elemento A escrito por T1, sin embargo, T1 se compromete después que T2, por lo que no es recuperable.

T1	T2
escribir(A) commit	leer(A) commit

15. ¿La planificación mostrada es recuperable?

Si

Se debe a que T2 lee los datos antes que T1, por lo que no depende de T1 y da igual que esta se comprometa después que T2.

T1	T2
escribir(A)	leer(A)
commit	commit

16. Una planificación recuperable sin cascada es aquella en la que, si T2 lee datos que ha escrito T1.

La transacción T1 se compromete antes que la operación de lectura de T2 |

La operación de escritura de T1 aparece antes de que T2 se comprometa |

La transacción T1 se compromete antes que la transacción T2 |

La operación de escritura de T1 aparece antes que la operación de lectura de T2

Por definición, una planificación recuperable sin cascada es aquella en la que, para todo par de transacciones T_i y T_j tales que T_j lee un elemento de datos escrito previamente por T_i , la operación comprometer de T_i aparece antes que la operación de lectura de T_j .

17. ¿La planificación mostrada es recuperable sin cascada?

No

Ya que, teniendo en cuenta la definición anterior, vemos que T1 se compromete después de la operación de lectura de T2, por lo que no es recuperable sin cascada a pesar de ser recuperable.

T1	T2
escribir(A)	leer(A)
commit	commit

18. ¿La planificación mostrada es recuperable sin cascada?

Si

Ya que, por la definición de planificación recuperable sin cascada, la transacción T1 se compromete antes de que se ejecute la operación de lectura de T2.

T1	T2
escribir(A)	
commit	leer(A)
	commit

19. Muchos sistemas de bases de datos se ejecutan, de forma predeterminada, en el nivel de aislamiento...

Lectura comprometida (leer datos comprometidos) |

Secuenciable (normalmente asegura la ejecución secuencial) |

Lectura repetible (comprometida y entre lecturas no hay actualización) |

Lectura no comprometida (permite leer datos no comprometidos)

Esto está así por defecto en muchas bases de datos, sin embargo, se puede especificar el nivel de aislamiento que interese para esa transacción.

20. ¿Qué nivel de aislamiento permite “escrituras sucias” (escribir datos escritos previamente antes por transacción no comprometida)?

Ninguno | Todos | Sólo lectura no comprometida | Solo lectura no repetible

En ningún tipo de nivel de aislamiento se permite escribir un elemento de datos que ya ha sido escrito por otra transacción que no se ha comprometido o abortado.

21. Las técnicas básicas de bloque sobre datos se pueden mejorar introduciendo dos tipos distintos de bloqueo.

Compartidos para lecturas y exclusivos para escrituras |
Compartidos para escrituras y exclusivos para lecturas |
Multiversión para lecturas e instantáneas para escrituras |
Multiversión para escrituras e instantáneas para lecturas |

Los bloqueos compartidos se usan para los datos que la transacción lee y los bloqueos exclusivos para aquellos que escribe. Varias transacciones pueden mantener bloqueos compartidos en el mismo elemento de datos a la vez, pero se concede un bloqueo exclusivo a una transacción sobre un elemento de datos solo si ninguna otra tiene un bloqueo. Este sistema permite la lectura concurrente de datos mientras aun se asegura la secuencialidad, evitando la escritura múltiple.

22. ¿Por qué el aislamiento de instantáneas es mejor, en términos de rendimiento, que las técnicas de bloqueo?

Porque los intentos de leer datos nunca tienen que esperar |
Porque las transacciones que sólo realizan lecturas se pueden abortar |
Porque la lectura de datos hace que las actualizaciones tengan que esperar |
Porque las técnicas de bloqueo pueden crear inconsistencias |

Lo que se consigue con la instantánea es darle a cada transacción su propia versión de la base de datos cuando comienza, siendo así esta su versión privada la cual nadie puede actualizar. Mientras se realiza la transacción, si esta modifica la base de datos, solo se actualiza en si versión. Si la transacción se compromete, entonces es cuando se guardan las modificaciones de la versión en la base de datos real. Consecuentemente, se pueden realizar varias transacciones de lectura a la vez ya que cada una estará trabajando con su propia versión.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Bases de Datos II

22-03-2021



Clase 05

Temas trabajados:

2. Gestión de Transacciones (Parte 3):

Control de Concurrency

Índice

1. Kahoot	3
-----------------	---

1. Kahoot

1. **¿Cuáles son los dos esquemas de control de concurrencia más utilizados?** Bloqueo en dos fases y Aislamiento de instantáneas | Bloqueo compartido y Bloqueo exclusivo | Protocolos basados en marcas temporales y Bloqueo en dos fases | Protocolos basados en validación y Aislamiento de instantáneas.
2. **Los protocolos basados en bloqueos se basan en que ninguna transacción puede modificar un dato mientras ...** otra previa puede modificar el mismo dato | no se actualiza la copia local de los datos | las marcas temporales no se modifican | la transacción previa no finalice.
Esto significa que se haga en exclusión mutua.
3. **Una transacción obtiene un bloqueo _____ sobre el dato X cuando puede leerlo pero no lo puede escribir.** compartido | exclusivo | compatible | secuenciable.
La definición de modo compartido es: si una transacción T_i obtiene un bloqueo en modo compartido (denotado por C) sobre el elemento Q, entonces T_i puede leer Q, pero no lo puede escribir. El modo que permite a T_i leer y escribir Q se llama exclusivo.
4. **El conjunto de reglas que indican cuando una transacción puede bloquear y desbloquear cada dato se denomina ...** protocolo de bloqueo | protocolo de consistencia | protocolo de fases | protocolo de concurrencia.
5. **¿Cuál es el orden correcto en la petición de un bloqueo? (T = Transacción; CC = Control de Concurrencia)** T pide bloqueo; CC concede bloqueo; T realiza operaciones | CC concede bloqueo; T pide bloqueo; T realiza operaciones | T pide bloqueo; T realiza operaciones; CC concede bloqueo | CC concede bloqueo; T realiza operaciones; T pide bloqueo
El control de concurrencia no es propositivo, es decir, no propone bloqueos. Lo que sucede es que la transacción pide el bloqueo y el control de concurrencia concede o no el bloqueo: si lo concede, la transacción realiza las operaciones; si no, la transacción puede hacer dos cosas: esperar y cuando se libere el bloqueo, se lo da (esto complica el control de concurrencia); o abortar, que es lo más sencillo.
6. **¿Qué modos de bloqueo, solicitados por las transacciones T1 y T2, son compatibles entre sí?** Compartido T1 y compartido T2 | Exclusivo T1 y compartido T2 | Compartido T1 y exclusivo T2 | Exclusivo T1 y exclusivo T2
7. **¿Es posible generar estados inconsistentes utilizando protocolos de bloqueo?** Sí, si los datos se desbloquean demasiado pronto | No, con estos protocolos siempre se generan estados consistentes
Si los datos se desbloquean demasiado pronto, pueden generar un problema, por lo que puede llevar a estados inconsistentes, que es lo peor que le puede pasar a una BD.
8. **Es verdad para el protocolo de bloqueo en dos fases.** Asegura la secuencialidad en cuanto a conflictos | Tiene una fase compartida seguida de una fase exclusiva | Las transacciones deben liberar bloqueos antes de obtener otros nuevos | Inicialmente la transacción entra en fase de decrecimiento
Es lo más importante, ya que genera estados consistentes de la BD.

9. ¿Qué transacción es de dos fases? T3 | T1 | T2 | T4

T1	T2	T3	T4
bloquear-c(A)	bloquear-c(A)	bloquear-x(A)	desbloquear(A)
bloquear-c(B)	leer(A)	leer(A)	desbloquear(B)
leer(A)	desbloquear(A)	bloquear-x(B)	leer(A)
leer(B)	bloquear-c(B)	leer(B)	leer(B)
A=A-50	leer(B)	A=A-50	A=A-50
B=B+50	desbloquear(B)	B=B+50	B=B+50
desbloquear(A)	A=A-50	escribir(A)	escribir(A)
desbloquear(B)	B=B+50	desbloquear(A)	escribir(B)
bloquear-x(A)	bloquear-x(A)	escribir(B)	bloquear-x(A)
bloquear-x(B)	escribir(A)	desbloquear(B)	bloquear-x(B)
escribir(A)	desbloquear(A)		
escribir(B)	bloquear-x(B)		
desbloquear(A)	escribir(B)		
desbloquear(B)	desbloquear(B)		

El protocolo de bloqueo en dos fases es: fase de crecimiento, donde se bloquean los datos necesarios; y una vez que termina esta fase, se pasa a la de decrecimiento, que es ir liberando. Una vez se entra en la fase de decrecimiento, ya no se pueden bloquear.

10. ¿Qué es un interbloqueo? Estado en el cual varias transacciones no pueden continuar su ejecución | Solicitud de bloqueo compartido por dos transacciones sobre el mismo dato | Solicitud de bloqueo exclusivo por dos transacciones sobre el mismo dato | Estado obtenido al aplicar el protocolo de bloqueo en dos fases

11. Los interbloqueos se pueden prevenir (no llegan a producirse) o detectar y recuperar (llegan y se resuelven). Verdadero

12. ¿Qué es preferible: un interbloqueo o un estado inconsistente? Interbloqueo | Estado inconsistente | Da igual

En una BD, cualquier cosa, por complicado que sea de gestionar, es mejor que un estado inconsistente.

13. ¿En qué protocolo del gestor de concurrencia se mantiene una copia de la base de datos para cada transacción? En los de aislamiento de instantáneas | En los basados en marcas temporales | En los basados en validación | En los de bloqueo en dos fases

Una transacción bajo el protocolo de bloqueo, cuando quiere un dato, tiene que pedir el bloqueo y si este está bloqueado, no puede seguir. En cambio, en el de instantáneas todos van hasta el final, cada uno con su copia local y terminan y se ejecutan, salvo que haya errores lógicos o de hardware.

14. En el esquema de control de concurrencia denominado aislamiento de instantáneas ... las transacciones de sólo lectura nunca son abortadas | las transacciones comparten copias locales de los datos | las transacciones de actualización de datos nunca son abortadas | las transacciones vuelcan su espacio de datos antes de comprometerse

Es una de las grandes ventajas, porque las transacciones de lectura nunca generan conflictos.

15. ¿Cuáles son las principales virtudes del aislamiento de instantáneas? Sobrecarga baja y sólo abortan transacciones que actualicen el mismo dato | Sobrecarga baja y sólo abortan transacciones que lean el mismo dato | Poca carga de datos en las copias locales y escaso uso de recursos hardware | Sobrecarga baja y poca carga de datos en las copias locales

16. Si dos transacciones entran en conflicto, la primera en obtener el bloqueo se compromete y realiza la actualización. Primera actualización gana | Primer compromiso gana | Primera instantánea gana | Primer atasco de escritura gana

Con cada transacción ocurre que llega, se le da su copia local y a trabajar. Pero llega el momento en el que están para terminar y el orden secuencial equivalente hay que garantizarlo.

“Primera actualización gana” se refiere a que la primera en obtener el bloqueo gana. En el caso de “primer compromiso gana” es cuando la transacción ha llegado al compromiso.

17. ¿Qué efecto hay que prevenir en el aislamiento de instantáneas? Actualizaciones perdidas | Fenómeno fantasma | Primer compromiso gana | Primera actualización gana

Actualización perdida: dos transacciones que se ejecutan concurrentemente podrían actualizar el mismo elemento de datos. Como ambas funcionan en aislamiento usando sus propias copias de la BD, ninguna ve las actualizaciones realizadas por la otra. Si a estas transacciones se les permite escribir en la BD, la primera actualización será sobrescrita por la segunda.

Fenómeno fantasma: es un fenómeno que ocurre cuando hay una transacción de solo lectura que llega en un momento muy conflictivo, es decir, cuando otra transacción está comprometiéndose y modificando el dato que va a leer.

Cuando se utiliza un sistema gestor de BD hay que saber cómo funciona su control de concurrencia. Y si es un aislamiento de instantáneas hay que tener cuidado con transacciones que lean datos, pero no los escriban o con inserciones de datos.

Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Base Datos II

05-04-2021



Clase 06

Temas trabajados: Tema 2 Parte 4, Gestión de
Transacciones, Sistemas de Recuperación

0.

1. Índice

2. Introducción	3
3. Kahoot	3

2. Introducción

Un lunes más, Mosquera nos sorprende con un kahoot y una clase magistral, en ese orden, en ocasiones, por no decir siempre, prescinde de la segunda. Para esta bitácora hemos recogido las preguntas del kahoot, las opciones y el razonamiento posterior sobre la pregunta.

3. Kahoot

1. ¿Una transacción con un error lógico puede volver a ejecutarse sin modificaciones?

Sí

No

Razonamiento Mosquera: Lo primero que hay que distinguir es entre los fallos físicos y los fallos lógicos. Un fallo físico es un fallo de daño en el sistema, pero un fallo lógico no significa que algo esté mal, por ejemplo, tengo una transacción que solicita al usuario 2 valores, y en un momento se divide el 1 valor entre el 2, si se introduce en el 2 se introduce un 0 se va a dar un error. Por lo que hay que contemplar si el error lógico tiene que ver con el planteamiento o con algo similar al ejemplo.

2. ¿Qué es el supuesto fallo-parada?

Los errores hardware/software no corrompen la memoria no volátil

Los errores hardware/software no corrompen la memoria volátil

Un bloque de disco puede perder su contenido en un fallo de disco

Un error lógico de una transacción puede corromper la memoria no volátil

Razonamiento Mosquera: Todos los sistemas informáticos actuales se diseñan bajo el concepto de supuesto fallo-parada, sistemas que ante errores hardware-software, todos tienen forma de que no se corrompa la memoria no volátil. No solo eso, sino que también se protegen, por ejemplo, una tarjeta que use condensadores, por si hay un corte de corriente que pueda salvarse los datos.

3. ¿Los algoritmos de recuperación realizan acciones durante el procesamiento normal de las transacciones?

Sí, para que exista información suficiente para permitir la recuperación.

Sí, para anticipar errores lógicos que pudieran provocar un fallo.

No, todo está disponible en la planificación de las transacciones.

No, la información está almacenada en la propia transacción.

3. Kahoot

Razonamiento Mosquera: Dicho de otra manera, los algoritmos de recuperación apuntan en la agenda los números de teléfono, mientras se está desarrollando una transacción los algoritmos de recuperación van anotando todos los datos, para que en caso de que ocurra algo podamos ir a esa “agenda” y recuperar los datos.

Hay que ser consciente de que esto en un sistema muy grande con cientos de miles de transacciones, hablamos de millones de escrituras, donde no es algo tan sencillo.

4. ¿Qué tipo de almacenamiento es esencial en los algoritmos de recuperación?

Almacenamiento estable

Almacenamiento volátil

Almacenamiento no volátil

Almacenamiento portátil

Razonamiento Mosquera: El almacenamiento estable es un concepto no un dispositivo. Podemos comprar una implementación mediante almacenamiento no volátil de un almacenamiento estable pero no tenemos algo físico.

5. La forma más básica de implementar almacenamiento estable es mediante...

...sistemas RAID de disco con imagen

... bloques físicos en memoria intermedia

... sistemas de control de concurrencia sin fallos

... transacciones RAID comprometidas

Razonamiento Mosquera: Pongo varios dispositivos de almacenamiento no volátil, replico en ellos la información y cuantas más copias tengo, más seguro tengo los datos.

Cuanto más discos tengamos es más caro, hay básicamente, dos implementaciones: Raid 2 o espejo y raid 5. El número corresponde a el número de copias realizadas.

6. Los sistemas más seguros de almacenamiento estable...

...guardan las copias duplicadas en espacios físicos diferentes

... realizan tres o más imágenes de disco

... combinan almacenamientos volátiles con almacenamientos no volátiles

... gestionan la información por bloques

3. Kahoot

Razonamiento Mosquera: Llevar la seguridad al máximo posible es que, no solo dupliquemos la información, sino que estén en sitios distintos para evitar daños externos (incendios, inundaciones...).

7. En un sistema de disco con imagen (RAID) la salida de datos (1 bloque lógico a 2 bloques físicos) está completa...

... cuando finaliza la segunda escritura

... cuando finaliza la primera escritura

... cuando finaliza la escritura del bloque físico 1

... cuando finaliza la escritura del bloque físico 2

Razonamiento Mosquera: Hay algunos sistemas de duplicación que tienen definido cual es el disco 1 y el 2, hay otro que no, que simplemente empiezan en uno y acaban en otro.

Se considera entonces cuando está copiado en el 2, ya que, si falla el sistema, sabemos que los datos están bien si las copias son iguales.

8. Las transacciones operan por medio de transferencias de datos entre...

... su área privada de trabajo y la memoria intermedia volátil

... su área privada de trabajo y el disco de almacenamiento estable

... la memoria intermedia volátil y el disco de almacenamiento estable

... su área privada y el disco no volátil

Razonamiento Mosquera: Una transacción funcionando en una base de datos solo trabaja con la memoria intermedia, no trabaja con el almacenamiento final, por motivos de eficacia.

Hay una consecuencia desde el punto de vista de los diseñadores, yo puedo tener una transacción, esta se ejecuta sin errores, se compromete, escribe todos los bloques de datos en la memoria intermedia, se ha comprometido no puede deshacerse. Sin embargo, no están los datos guardados, ¿qué pasa si el sistema falla? Pues es algo que hay que tener en cuenta en el sistema de recuperación.

9. Para conseguir el objetivo de la atomicidad de una transacción...

... se almacena la información sobre las modificaciones antes de realizarlas

... se realizan las modificaciones antes de almacenar su información

... se realizan las modificaciones sobre memoria no volátil

3. Kahoot

... se realizan las modificaciones sobre memoria volátil

Razonamiento Mosquera: Si hago una modificación en una memoria no volátil ya es estable (que es una de las respuestas incorrectas), pero es más importante almacenar información sobre las modificaciones que la modificación en sí.

10. ¿Dónde debe residir el registro histórico de operaciones?

Almacenamiento no volátil

Almacenamiento estable

Almacenamiento volátil

Almacenamiento portátil

*El almacenamiento no volátil es el aparato físico.

Razonamiento Mosquera: cuando hablamos de almacenamiento estable nos referimos a construir a partir de almacenamientos no volátiles una estructura que no sea sensible a fallos

¿Cada vez que una transacción hace una operación tengo que acceder al almacenamiento no volátil que implementa el almacenamiento estable y registrarlo? Es decir, todo lo ganado en eficiencia en la respuesta no haciendo las actualizaciones de datos en la memoria de almacenamiento estable, sino en la volátil (a riesgo de perderlos), ¿ahora para el registro histórico hay que escribir todo en almacenamiento estable?

El registro histórico se va volcando en almacenamiento volátil (a riesgo de perderlo), y cuando el bloque de datos que está en almacenamiento volátil se va a almacenamiento no volátil, el registro histórico correspondiente se va a almacenamiento estable.

Cuando hablamos de registro histórico hablamos de varios registros. El más importante es el de actualización.

11. En el registro de actualizaciones de datos del registro histórico NO se almacena

El identificador de la transacción

El valor anterior del dato

El estado de la transacción (iniciada, comprometida, abortada)

El valor nuevo del dato

Razonamiento Mosquera: hablamos de registro histórico, pero sería más correcto hablar de registros históricos ya que este último concepto son varios registros diferentes, y uno de ellos (el más importante) es el registro de actualización. Este nos dice:

3. Kahoot

Qué transacción ha actualizado qué dato desde el valor anterior x al valor actual y . De esta forma se tiene en cuenta el valor origen y destino ya que, si eso no se ha grabado:

- a. Una transacción comprometida dejó los datos el mismo día en intermedio y no se grabaron en principal, cojo el registro histórico, repaso las actualizaciones de dicha transacción que está comprometida y vuelvo a grabar eso en la memoria secundaria, en la intermedia, y ya se grabará en el estable.
- b. Una que “me ha pillado en el medio” y hay que deshacer cosas, voy cogiendo valores actuales y voy cambiándolos por valores anteriores, dando pasos hacia atrás en el proceso.

Existe un registro de estado de transacciones. Para que yo sepa si en función de si estaba iniciada, comprometida o abortada, cómo tengo que ir al registro de actualizaciones y seguir el camino de avanzar o de retroceder.

12. Se dice que una transacción modifica la base de datos si realiza una actualización

Exclusivamente en almacenamiento estable

Exclusivamente en la memoria intermedia volátil

En la memoria intermedia volátil o en el disco no volátil

Exclusivamente en el disco no volátil

Razonamiento Mosquera : sirven los tres tipos de almacenamiento. Es una propiedad importante para que sea compatible con el sistema de concurrencia.

Si es una actualización (generan conflictos en las concurrencias, a diferencia de las lecturas) cuando una transacción modifica memoria intermedia, tengo que hacer que sea compatible con el sistema de concurrencia.

Se habla de dos tipos de modificaciones: la inmediata (va a volcar) y la diferida (termina y luego vuelca todo). En cuanto a la compatibilidad con los sistemas de concurrencia:

- El aislamiento de instantáneas encaja muy bien con la modificación diferida: yo le doy en memoria intermedia una zona privada a la transacción y cuando termina completa se vuelva si el sistema de concurrencia me lo permite (este último ha hecho un bloqueo por el método de aislamiento instantáneo).
- La inmediata funciona muy bien con el bloqueo en dos fases: en la segunda fase de desbloqueo, cuando voy soltando los bloqueos exclusivos, puedo ir grabando en memoria intermedia, entonces ambos son compatibles.

Es importante tener esto en cuenta para ver la compatibilidad de mi sistema de recuperación con el de concurrencia, que ambos deben funcionar de forma sólida.

13. El registro histórico permite implementar las siguientes operaciones

Borrar (recuperar valor anterior) e Insertar (especificar valor nuevo)

Entrada (recuperar valor anterior) y salida (especificar valor nuevo)

Deshacer (recuperar valor anterior) y rehacer (especificar valor nuevo)

Leer (recuperar valor anterior) y escribir (especificar valor nuevo)

Razonamiento Mosquera: como se dijo antes, el registro histórico nos permite ir hacia delante o volver hacia atrás. Completar algo que no se ha grabado, o deshacer algo que se ha hecho parcialmente (son las dos operaciones posibles).

14. Se dice que una transacción es comprometida cuando...

...todo su registro histórico se ha guardado

...realiza la última actualización de la base de datos

...realiza su última operación de escritura

...todo su registro histórico se ha guardado en almacenamiento estable

Razonamiento Mosquera: cuando se definía a nivel de propiedades ACID, se hablaba de comprobar el diagrama de transiciones: la transacción y los diversos estados: iniciada, comprometida, comprometida parcialmente, abortada. Se hablaba de comprometida cuando terminaban, no solo las operaciones, sino la última operación de escritura.

En realidad, la última operación de escritura, como hemos visto antes, una actualización de una transacción en la base de datos también puede ser en memoria intermedia, y lo que está en memoria intermedia se puede perder. Hay que fijarse en la definición que sale en el libro, se habla de transacción comprometida NO cuando los datos están en almacenamiento estable, sino cuando el registro histórico está en almacenamiento estable. Si cuando se va a hacer una operación, la que sea, primero se registra el dato en el registro histórico, y luego se hace, siempre hay una posibilidad de que después de guardar en el registro histórico, y antes de hacer la operación (en esos milisegundos que transcurren) se produzca un fallo, Lo importante es el almacenamiento del registro histórico. Con los datos en el registro la transacción se puede dar por finalizada, aunque de forma efectiva la última operación no se haya ejecutado ya que, el registro histórico en caso de fallo, permite recuperar esa última operación. Sin embargo, si no lo hubiera guardado en el registro histórico, la operación no sería recuperable.

Por eso incluso a nivel de almacenamiento es más importante guardar bien el registro histórico que los propios datos, por todo lo explicado anteriormente.

15. Los puntos de revisión permiten...

- ...asegurar el volcado de la memoria intermedia en el disco
- ...ralentizar el procesado de transacciones
- ...reducir la carga que supondría la lectura completa del registro histórico
- ...evitar un fallo en memoria estable

Razonamiento Mosquera: cuando voy guardando el registro histórico en el conjunto de registro histórico (con todos sus subregistros, que no hemos visto prácticamente ninguno), voy acumulando en una base de datos “normalita” miles y decenas de miles de entradas. Si falla el sistema, ¿leo todo? No tendría sentido, ya que hay transacciones de hace por ejemplo dos semanas que han terminado y no han dado problema. Lo que se hace antes es marcar puntos de revisión y se parte desde ahí, reduciendo la carga que supondría la lectura completa.

16. NO está permitido durante la operación de establecimiento de un punto de revisión

- Escritura en almacenamiento estable del registro histórico
- Escritura en disco de los bloques modificados en memoria intermedia
- Actualización de base de datos o de registro histórico por transacciones
- Actualización del registro histórico con la lista de transacciones activas

Razonamiento Mosquera: las tres incorrectas son precisamente lo que hago:

Escritura en almacenamiento estable del registro histórico: para saber que lo tengo sin partes aún en la memoria intermedia.

Escritura en disco de los bloques modificados en memoria intermedia: si tengo algo en memoria intermedia que todavía no he copiado, lo escribo al disco.

Actualización del registro histórico con la lista de transacciones activas: cuando se hace un punto de revisión no se paran las transacciones, solamente se bloquean las actualizaciones. De este modo puede haber transacciones que están ejecutándose, pero están esperando por una autorización para hacer una actualización, o puede haber otras que solo son lectura que siguen adelante sin problema. Lo que no puede pasar mientras estoy haciendo la foto (en los puntos de revisión) es que lo que está en la foto se mueva. Lo que está en la foto son los datos de la base de datos que se encuentran en memoria intermedia y el registro histórico. No puedo permitir que esas dos cosas de las que estoy haciendo una foto, la cual voy a llevar a almacenamiento estable, se muevan. NO permito actualizaciones, lecturas sí. Tener puntos de revisión de vez en cuando resulta muy útil para limpiar el registro histórico: tengo un fallo, miro el registro histórico desde

3. Kahoot

abajo y busco el último punto de revisión. Sé que en ese punto de revisión he llevado a almacenamiento estable al registro histórico y los datos. Yo tengo que ver a partir de ahí (tengo transacciones activas) y subir un poco más para ver dónde empezaron las transacciones activas y llegar lo suficientemente arriba como para que la primera también esté en el registro que voy a recuperar, porque voy a rehacer las operaciones.

17. ¿Es lo mismo almacenamiento estable que almacenamiento no volátil?

No

Sí

Razonamiento Mosquera: estable es una implementación mediante varios almacenamientos no volátiles de sistemas de seguridad que, ante fallo de un volátil, no se pierde la información.

18. ¿Qué tarea NO se realiza durante un volcado de una base de datos?

Copiar el contenido de la base de datos en almacenamiento estable

Tener el mayor número posible de transacciones activas

Escribir en almacenamiento estable el registro histórico

Escribir en disco no volátil los bloques de la memoria intermedia volátil

Razonamiento Mosquera: las respuestas incorrectas son prácticamente las mismas de cuando hablábamos de establecer los puntos de revisión:

- Coger lo que tengo en memoria intermedia (volátil)
- Llevarla a no volátil
- Si por duplicación aplico las técnicas para conseguir almacenamiento estable, aplicarlas para que todo esté en estable.

La diferencia con el punto de revisión que establecíamos antes es que permitíamos que las transacciones siguieran activas, PERO sin hacer actualizaciones. De forma que cuando teníamos que volver atrás, íbamos al punto de revisión, pero debíamos retroceder un poco para ver dónde comenzaban las transacciones que están activas durante el punto de revisión. En el caso del volcado simplemente paramos el sistema de transacciones: dejamos que las que se están ejecutando terminen sin admitir ninguna nueva y, cuando no quede ninguna en ejecución, hacemos el volcado en almacenamiento estable.

19. Con qué temporalidad se debería establecer un punto de revisión en el RH y/o realizar un volcado de la BD?

Cada hora

Cada semana

Depende

Cada día

Razonamiento Mosquera: cuantas más veces se haga, mayor garantía de seguridad en la base de datos, pero hará falta más tiempo de computación. Es muy habitual cada día (no es una norma) sobre todo en horas bajas de tráfico de operaciones (por la noche, por ejemplo).

En sistemas internacionales no es tan fácil utilizar ese criterio ya que las horas de la noche no coinciden para todos los países. Estos utilizan técnicas para hacerlo sin parar el sistema. Estos van a requerir sistemas de duplicación de información más amplios y, por lo tanto, más caros.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Bases de Datos II

12-04-2021



Clase 07

Temas trabajados:

3. SQL

Índice

1. Vistas	3
2. Transacciones	4
3. Autorización	5
Concesión y revocación de privilegios	5
Roles	6
Autorización sobre vistas	6
Autorizaciones sobre esquemas	6
Transferencia de privilegios	7
Revocación de privilegios	7
4. Funciones y procedimientos	7
Declaración e invocación de funciones y procedimientos de SQL	7
Construcciones del lenguaje para procedimientos y funciones	8
Rutinas en otros lenguajes	8
5. Disparadores	9
Necesidad de los disparadores	9
Los disparadores en SQL	9
Cuando no deben emplearse los disparadores	10
6. Seguridad de las aplicaciones	11
Inyección SQL	11
Falsificación de solicitudes y ejecuciones de comandos entre webs	11
Filtrado de contraseñas	12
Autenticación de aplicación	12
Autorización a nivel de aplicación	13
Trazas de autoría	13
Privacidad	14
7. Cifrado y sus aplicaciones	14
Técnicas de cifrado	14
Cifrado soportado en las bases de datos	15
Cifrado y autenticación	16

1. Vistas

Una **vista** es una relación *virtual* definida mediante una consulta, y que conceptualmente contiene las tuplas que son resultado de dicha consulta. Las tuplas de las vistas no se almacenan en la BBDD: cuando se hace referencia a una vista se ejecuta la consulta asociada. Por decirlo de otra manera, las vistas son a las relaciones lo que los atributos calculados a los atributos. Por esto, las vistas son creadas cuando se necesiten, bajo demanda.

La principal ventaja que presentan es no tener que reescribir la consulta todas las veces que se necesite, simplemente se haría referencia a la vista. También tienen un uso enfocado a la seguridad.

En SQL una vista se define mediante el comando **create view** de la siguiente forma:

```
create view v as <query expression>;
```

siendo v el nombre de la vista a crear y <query expression> cualquier consulta válida.

Las vistas pueden definir los nombres de sus atributos de forma explícita, por ejemplo:

```
create view departments_total_salary(dept_name, total_salary) as
    select dept_name, sum (salary)
    from instructor
    group by dept_name;
```

En cualquier otro contexto de SQL se usan como si de una relación/tabla se tratase.

Existe un tipo de vistas denominadas **vistas materializadas**, que sí se almacenan en la BBDD. El problema de estas vistas es la actualización, ya que pueden contener datos obsoletos. El proceso de mantener la vista materializada actualizada se llama mantenimiento de vista materializada (o simplemente mantenimiento de vista). Hay varias formas de realizar este mantenimiento:

- Cuando cualquiera de las relaciones implicadas en ella se modifica.
- Cuando se usa la vista.
- Periódicamente.

Elegir un método u otro dependerá del sistema de la base de datos y, en caso de que el sistema permita elegir entre varios, del administrador de la base de datos y de sus preferencias. Las vistas materializadas son ventajosas en aquellas aplicaciones donde se usa frecuentemente una vista determinada o donde se requiere un tiempo de respuesta mínimo, evitando así la computación de la vista todas las veces que ésta se solicita. De todos modos, hay que observar si los beneficios son mayores que el almacenamiento extra a usar y el coste del mantenimiento de vistas.

A pesar de que son útiles para consultas, las vistas presentan problemas serios en cuanto a inserción, actualización y borrado de datos. Estas modificaciones no están generalmente permitidas en las vistas, salvo casos limitados. En SQL se dice que una vista es actualizable si cumple, en la consulta relacionada a la misma, todas las siguientes condiciones:

- La cláusula **from** tiene solo una relación de la base de datos.
- La cláusula **select** contiene sólo nombres de atributos de la relación, y no tiene ninguna expresión, agregación, o especificación **distinct**.
- La consulta no tiene las cláusulas **group by** y **having**.

Aún con estas condiciones puede haber inserciones en una vista que no satisfagan la cláusula **where** de la misma, siendo, así, una inserción inconsistente (no aparecerá posteriormente en la vista). Para resolver esto se puede añadir al final de la definición de la vista la cláusula **with check option**, y, si una tupla insertada no satisface la condición o condiciones de la cláusula **where** de la vista, esa inserción se rechaza. Con las actualizaciones pasa algo similar: se rechaza el nuevo valor si no satisface la condición o condiciones de la cláusula **where**.

2. Transacciones

Una **transacción** consiste en una secuencia de instrucciones de consulta o de actualización. Una transacción comienza cuando se ejecuta una sentencia de SQL. Una de las siguientes sentencias SQL debe finalizarla:

- **Commit work** (compromiso): compromete la transacción actual, es decir, hace que las actualizaciones realizadas por la transacción pasen a ser permanentes en la BBDD. Una vez comprometida, se inicia automáticamente una nueva.
- **Rollback work** (retroceso): provoca el retroceso de la transacción actual, es decir, deshace sus actualizaciones. Por tanto, el estado de la BBDD se restaura al que tenía antes de la ejecución de la 1ª instrucción de la transacción.

La palabra clave **work** es opcional en ambas instrucciones.

El retroceso es útil si se detecta algún error durante la ejecución. El compromiso es parecido, en cierto sentido, a guardar los cambios de un documento que se esté editando, mientras que el retroceso es como abandonar la sesión de edición sin guardar cambios. Una vez se ha ejecutado **commit work**, sus efectos ya no se pueden deshacer con **rollback work**. El sistema de BBDD garantiza en caso de fallo el retroceso de los efectos de la transacción si todavía no se había ejecutado **commit work**.

Ya sea comprometiendo las acciones de una transacción tras haberse completado, o retrocediendo todas sus acciones en el caso de que no se pueda completar entera, la BBDD proporciona una abstracción de que la transacción es **atómica**, es decir, indivisible. O bien se ven reflejados todos sus efectos o no se ve ninguno.

Si un programa termina sin ejecutar ninguno de los comandos, las actualizaciones se comprometen o se retroceden. La norma no especifica cuál de las 2 opciones tiene lugar, con lo que la elección depende de la implementación.

En muchas implementaciones de SQL, cada instrucción de SQL se considera de manera predeterminada como una transacción en sí misma, y se compromete en cuanto se ejecuta. El

compromiso automático se puede desactivar si hay que ejecutar una transacción que consta de varias instrucciones SQL. La forma de desactivación depende de cada implementación de SQL.

Una alternativa mejor es permitir que se encierren varias instrucciones SQL entre las palabras clave **begin atomic ... end**. Todas las instrucciones entre esas palabras clave forman así una única transacción.

3. Autorización

Se pueden asignar a los usuarios varios tipos de autorizaciones para diferentes partes de la BBDD: de lectura de datos, de inserción de nuevos datos, de actualización de datos y de borrado de datos.

Cada uno de estos tipos de autorización se llama **privilegio**. Los usuarios pueden obtener una, ninguna o varias autorizaciones. También reciben concesiones de autorización sobre los esquemas de la BBDD, permitiéndoles, por ejemplo, crear, modificar o eliminar relaciones. Un usuario puede conceder su autorización a otro o la revoke.

El último elemento de autoridad es el que se le da al administrador de la BBDD. Este puede autorizar a nuevos usuarios, reestructurar la BBDD y mucho más.

Concesión y revocación de privilegios

La norma de SQL incluye los privilegios **select**, **insert**, **update** y **delete**. El privilegio **all privileges** se utiliza como forma abreviada de todos los privilegios que se pueden conceder. El usuario que crea una relación recibe todos los privilegios sobre esa.

El lenguaje de definición de datos de SQL incluye comando para conceder y revocar privilegios. La instrucción **grant** se utiliza para conceder autorizaciones.

```
grant <lista de privilegios>
on <nombre de relación o de vista>
to <lista de usuarios o de roles>;
```

La lista de privilegios permite conceder varios privilegios en un solo comando.

La autorización **select** sobre una relación es necesaria para leer las tuplas de una relación. La autorización **update** permite a los usuarios actualizar cualquier tupla de la relación, esta puede concederse sobre todos los atributos o solo algunos. La autorización **insert** sobre una relación permite a los usuarios insertar tuplas en la relación, también puede especificar una lista de atributos. La **delete** permite borrar tuplas.

Es importante tener en cuenta que el mecanismo de autorización de SQL concede privilegios sobre una relación completa o sobre determinados atributos de una relación. Sin embargo, no permite la autorización sobre tuplas concretas de una relación.

Para revocar se usa **revoke**:

```
revoke <lista de privilegios>  
on <nombre de relación o de vista>  
to <lista de usuarios o de roles>;
```

La revocación de privilegios resulta más compleja si el usuario al que se le revocan los privilegios se los ha concedido a otros usuarios.

Roles

Se crea un conjunto de roles en la BBDD. Las autorizaciones se conceden a los roles de la misma forma que se conceden a los usuarios individuales de la misma forma que se conceden a los usuarios individuales. A los usuarios de la BBDD se les concede un conjunto de roles que los usuarios pueden realizar.

Los privilegios de un usuario o un rol constan de:

- Todos los privilegios que se concedan directamente al usuario o al rol.
- Todos los privilegios que se concedan a los roles que se hayan concedido al usuario o al rol.

Hay que tener en cuenta que el concepto de autorización por rol no es específico de SQL y se usa para el control de una gran variedad de aplicaciones compartidas.

Autorización sobre vistas

Un usuario que crea una vista no necesariamente recibe todos los privilegios sobre dicha vista, solo recibe aquellos que no proporcionan autorización adicional más allá de los que ya tiene. Por ejemplo, el usuario que crea una vista no puede obtener una autorización **update**. Si un usuario crea una sobre la que no se le puede conceder ninguna autorización, el sistema denegará la petición de creación.

El privilegio **execute** se puede conceder a una función o a un procedimiento, permitiendo a un usuario ejecutar la función o procedimientos.

Si la definición de la función tiene una cláusula extra **sql security invoker**, entonces se ejecuta con los privilegios del usuario que la invoca, en lugar de con los privilegios de quien define la función. Esto permite la creación de bibliotecas de funciones.

Autorizaciones sobre esquemas

La norma de SQL solamente deja al propietario del esquema realizar modificaciones en el mismo (crear o borrar relaciones, añadir y eliminar atributos y añadir o eliminar índices).

Sin embargo, SQL incluye un privilegio de referenciar (**references**), que permite que un usuario declare claves externas cuando crea relaciones. El privilegio se concede sobre atributos concretos, como el **update**.

Evitar que los usuarios creen claves externas que hagan referencia a otra relación favorece a que no se restrinjan actividades futuras a otros usuarios.

También es necesario para crear restricciones **check** sobre una relación *r*, si esta tiene una subconsulta referenciándola.

Transferencia de privilegios

A un usuario al que se le haya concedido algún tipo de autorización puede permitírsele trasladar esta autorización a otros usuarios. Un privilegio o rol no se puede trasladar, si se desea hacerlo se añade la cláusula **with grant option**.

El paso de una determinada autorización de un usuario a otro se puede representar mediante un **grafo de autorización**. Los nodos del grafo son los usuarios. Un usuario tiene una autorización si, y sólo si, existe un camino desde la raíz del grafo de autorización hasta el nodo que representa al usuario.

Revocación de privilegios

La revocación de un privilegio de un usuario o rol puede causar que otros usuarios o roles también lo pierdan. Este comportamiento se denomina revocación en cascada. En la mayoría de los sistemas de BBDD esta cascada es el comportamiento predeterminado. Sin embargo, la sentencia **revoke** puede especificar la cláusula **restrict** para evitar que se produzca una revocación en cascada.

Se puede usar la palabra clave **cascade** en lugar de **restrict** para indicar que la revocación se produce en cascada; sin embargo, se puede omitir.

La revocación en cascada no es apropiada en muchas situaciones.

Para conceder un privilegio con un conjunto de concesiones al rol actual asociado a una sesión, se puede añadir la cláusula: **granted by current_role** a la sentencia de concesión, siempre que el rol actual no sea null.

4. Funciones y procedimientos

Las funciones resultan particularmente útiles con los tipos de datos especializados como las imágenes y objetos geométricos. De hecho, un tipo de dato segmento utilizado en una BBDD de mapas puede tener una función asociada que comprueba si 2 segmentos se superponen, y un tipo de datos imagen puede tener funciones asociadas para comparar las similitudes entre 2 imágenes.

Las funciones y los procedimientos permiten almacenar la “lógica de negocio” en la BBDD y ejecutarla con sentencias de SQL.

Declaración e invocación de funciones y procedimientos de SQL

La norma SQL soporta funciones que pueden devolver tablas como resultado; esas funciones se llaman **funciones de tabla**. En general, las funciones evaluadas sobre tablas pueden considerarse **vistas parametrizadas** que generalizan el concepto habitual de las vistas permitiendo la introducción de parámetros.

Se pueden invocar los procedimientos mediante la sentencia **call** desde otro procedimiento de SQL. SQL permite varios procedimientos con el mismo nombre, si los argumentos son distintos.

Construcciones del lenguaje para procedimientos y funciones

SQL soporta varias construcciones que le proporcionan casi toda la potencia de los lenguajes de programación de propósito general. La parte de la norma SQL que trata esto se llama **módulo de almacenamiento persistente**.

Las variables se declaran utilizando la sentencia **declare** y puede tener cualquier tipo válido de SQL. Las asignaciones se realizan mediante la sentencia **set**.

Las sentencias compuestas son de la forma **begin... end**, y pueden contener varias sentencias de SQL entre begin y end. En ellas se pueden declarar variables locales. Una sentencia compuesta de la forma **begin atomic... end** asegura que todas las sentencias se ejecutan como una única transacción.

SQL soporta las sentencias **while, repeat, for, iterate, if-then-else** y **case**. También soporta la señalización de las **condiciones de excepción** y la declaración de **manejadores** que pueden tratar esa excepción.

Las sentencias entre **begin y end** pueden provocar una excepción ejecutando **signal**. El manejador dice que si se da la condición, la acción que hay que tomar es salir de la sentencia begin end circundante. Una acción alternativa sería **continue**, que continúa con la ejecución a partir de la siguiente instrucción que ha generado la excepción. Además de las condiciones definidas de manera explícita, también hay condiciones predefinidas como **sqlexception, sqlwarning** y **not found**.

Rutinas en otros lenguajes

SQL permite definir funciones en lenguajes de programación como Java, C#, C o C++. Las funciones definidas de esta manera pueden ser más eficientes que las definidas en SQL, y los cálculos que no pueden llevarse a cabo en SQL pueden ejecutarse mediante estas funciones.

Los procedimientos del lenguaje externo tienen que tratar con valores nulos (tanto **in** como **out**) y con valores de retorno. También necesitan comunicar el estado de éxito/error, para tratar con las excepciones. Esta información se puede comunicar mediante parámetros extra: un valor **sqlstate** para indicar el éxito/fracaso. También se pueden utilizar otros mecanismos para manejar los valores null. Sin embargo, si una función no trata con estas situaciones, se puede añadir una línea extra **parameter style** general a la declaración anterior indicando que los procedimientos o las funciones externos solo toman los argumentos mostrados y no tratan con valores nulos ni excepciones.

Cuando el código está escrito en un lenguaje “seguro” como Java se puede ejecutar en un **arenero**, que permite que el código tenga acceso a su propia área de memoria.

5. Disparadores

Un **disparador** es una sentencia que el sistema ejecuta de manera automática como efectos secundarios de la modificación de la BBDD. Para diseñar un mecanismo disparador se deben cumplir 2 requisitos:

1. Especificar las condiciones en las que se va a ejecutar el disparador. Se descompone en un evento que provoca la comprobación del disparador y una condición que se debe cumplir para que se ejecute el disparador.
2. Especificar las acciones que se van a realizar cuando se ejecute el disparador.

Una vez se introduce un disparador en la BBDD, el sistema asume la responsabilidad de ejecutarlo cada vez que se produzca el evento especificado y se satisfaga la condición correspondiente.

Necesidad de los disparadores

Los disparadores se pueden utilizar para implementar ciertas restricciones de integridad que no se pueden especificar utilizando el mecanismo de restricciones de SQL. También son útiles para alertar a los usuarios o para iniciar de manera automática ciertas tareas cuando se cumplen determinadas condiciones.

En general, los sistemas de disparadores no pueden realizar actualizaciones fuera de la BBDD.

Los disparadores en SQL

La cláusula **referencing new row as** crear una variable fila nueva (denominada **transition variable**), que almacena el valor de la fila insertada después de la inserción.

Para usar los disparadores hay que asegurar la integridad referencial en las inserciones, borrados y actualizaciones.

Para las actualizaciones, el disparador puede especificar los atributos cuya actualización provoca su ejecución; las actualizaciones de otros atributos no causarán que se ejecute.

La cláusula **referencing old row as** se puede utilizar para crear una variable que almacene el valor anterior de una fila actualizada o borrada. La cláusula **referencing new row as** se puede usar con las actualizaciones además de con las inserciones.

Muchos sistemas de BBDD soportan otros muchos eventos disparadores, como cuando un usuario se registra en la BBDD, si el sistema se apaga o si se realizan cambios en la configuración del sistema.

Los disparadores se pueden activar antes (**before**) del evento (**insert, delete o update**), pudiendo servir como restricciones adicionales que pueden impedir actualizaciones, inserciones o borrados no válidos; en lugar de después (**after**) del evento.

Los disparadores se pueden usar también para sustituir un valor por un null. Con la sentencia set se realizan estas modificaciones.

En lugar de llevar a cabo una acción por cada fila afectada, se puede realizar una sola acción para toda la sentencia de SQL que ha causado la inserción, el borrado o la actualización. Para hacerlo se utiliza la cláusula **for each statement** en lugar de **for each row**. Las cláusulas **referencing old table as** o **referencing new table as** se pueden emplear para hacer referencia a tablas temporales

(tablas de transición) que contengas todas las filas afectadas. Estas no se pueden emplear con los disparadores **before**, pero sí con los **after**. Se puede usar una única sentencia de SQL para llevar a cabo varias acciones con base en las tablas de transición.

Los disparadores se pueden habilitar y deshabilitar; de manera predeterminada, al crearlos se habilitan, pero se pueden deshabilitar empleando **alter trigger nombre_disparador disable**. Los disparadores también se pueden eliminar: **drop trigger nombre_disparador**.

Cuando no deben emplearse los disparadores

Existen muchas aplicaciones para los disparadores, pero algunas se manejan mejor con técnicas alternativas. Por ejemplo, se puede implementar la característica **on delete cascade** de una restricción de clave externa mediante un disparador, en lugar de la característica en cascada. No solo implicaría un mayor trabajo de implementación, también sería mucho más difícil para el usuario de la BBDD entender el conjunto de restricciones que implementa la BBDD.

También se pueden utilizar para mantener las vistas materializadas. Sin embargo, muchos sistemas de BBDD actuales soportan las vistas materializadas, por lo que, no serían necesarios los disparadores.

Los disparadores también se usan para manipular copias de la BBDD. Se puede crear una colección de disparadores sobre la inserción, borrado o actualización de cada relación para registrar las modificaciones en relaciones llamadas **cambio** o **delta**. Los sistemas de BBDD proporcionan características predeterminadas para la réplica de las BBDD, lo que hacen que los disparadores resulten innecesarios en algunos casos.

Otro problema es la ejecución no pretendida de la acción disparada cuando se cargan datos de una copia de seguridad, o cuando las actualizaciones de la BBDD de un sitio se replican en un sitio de copia de seguridad. En estos casos, el disparador ya se ha ejecutado y no debe volverse a ejecutar. Al cargar los datos, los disparadores se pueden deshabilitar de manera explícita. Para los sistemas de réplica de copia de seguridad que pueden que tengan que sustituir el sistema principal, hay que deshabilitar los disparadores en un principio y habilitarlos cuando el sitio de copia de seguridad sustituye al principal en el procesamiento. Como alternativa, algunos sistemas permiten que los disparadores se especifiquen como **not for replication**, que garantiza que no se ejecuten en el sitio de copia durante la réplica. Otros sistemas ofrecen una variable del sistema que denota que la BBDD es una réplica en la que se están repitiendo las acciones de la BBDD.

Los disparadores se deben escribir con sumo cuidado. Los disparadores pueden utilizarse para objetivos muy útiles, pero es preferible evitarlo si existen alternativas.

6. Seguridad de las aplicaciones

La seguridad de la aplicación tiene que lidiar con amenazas más allá de las tratadas con la autorización SQL. Las aplicaciones deben autenticar usuarios y asegurarse de que esos usuarios sólo pueden realizar las tareas para las que están autorizados. Hay varios caminos mediante los cuales la seguridad de una aplicación se puede ver comprometida.

Inyección SQL

El atacante logra que la aplicación ejecute una consulta SQL creada por él. Imaginemos una aplicación que tiene la siguiente sentencia en su lenguaje anfitrión (Java en este caso):

```
String query = "select *
                from student
                where name like '%"
                + name + "%'";
```

donde `name` es un string introducido por el usuario.

En el formulario donde se introduce el nombre, el atacante puede poner “”; <alguna consulta SQL>; --”. El texto insertado hace que la consulta quede de la siguiente forma:

```
select * from student where name like ‘’; <alguna consulta SQL>; --’
```

De esta forma, el texto insertado cierra la cláusula **where** y el intérprete ejecuta las siguientes consultas, creadas por el atacante. De esta manera se puede obtener información sensible de la BBDD y, por tanto, de la aplicación.

Para evitar este tipo de ataques, los lenguajes anfitriones tienen los llamados **prepared statements** (declaraciones preparadas).

Falsificación de solicitudes y ejecuciones de comandos entre webs

Un sitio web que permite introducir texto a los usuarios (como un comentario o nombre), almacenarlo y posteriormente mostrárselo a otros usuarios es potencialmente vulnerable a un tipo de ataque llamado **cross-site-scripting** (XSS, ejecuciones de comandos entre webs). En este tipo de ataques, el atacante inserta código escrito en un lenguaje del lado del cliente (como JavaScript o Flash) en lugar de insertar el comentario o nombre solicitado. Cuando otro usuario visualiza el texto insertado, el navegador podría ejecutar el script, que puede realizar acciones como enviar información privada de cookies al atacante, o ejecutar una acción en un sitio web diferente en el cual el usuario víctima podría tener la sesión iniciada. Este tipo de vulnerabilidad también se llama **cross-site request forgery** (XSRF o CSRF, falsificación de solicitudes entre webs).

Para protegerse contra este tipo de ataques hay dos cosas que hacer:

- **Prevenir el sitio web de ser usado como lanzador de ataques XSS/XSRF.** La técnica más simple es desactivar cualquier tag HTML en la entrada de texto por parte de los usuarios. Hay funciones que detectan y quitan todas estas etiquetas. Estas funciones pueden usarse para prevenir tags HTML y como resultado ningún script puede ser mostrado a otros usuarios.

- **Prevenir el sitio web de ataques XSS/XSRF lanzados desde otras webs.** Si tienes sesión iniciada en una web y visitas otra vulnerable a ataques XSS, la ejecución del código malicioso en el navegador del usuario puede ejecutar acciones en el sitio web en el que tienes la sesión iniciada, o pasar información relacionada con la sesión al sitio web y llegar al atacante. Esto no se puede prevenir por completo, pero sí minimizar los riesgos.
 - El protocolo HTTP permite un servidor para comprobar la referencia a la página web accedida (URL). Comprobando esto, ataques XSS originados en otras páginas webs pueden ser prevenidos.
 - En lugar de usar la cookie para identificar la sesión, la sesión puede también ser restringida a una dirección IP. Como resultado, aunque el atacante obtenga la cookie, no va a tener permitido iniciar sesión desde otro ordenador.

Filtrado de contraseñas

Otro problema de los desarrolladores de aplicaciones es lidiar con el almacenamiento de contraseñas. Muchos servidores de aplicaciones ofrecen mecanismos para almacenar contraseñas cifradas, que el servidor descifra antes de pasarlas a la base de datos. Estos mecanismos eliminan la necesidad de almacenar las contraseñas como texto claro en los programas de aplicaciones. Sin embargo, si la clave de descifrado es vulnerable a ser expuesta, esto no es completamente efectivo.

Otra medida que ayuda a proteger las contraseñas en las BBDD es que muchos sistemas de BBDD permiten el acceso a la base de datos a un conjunto dado de direcciones IP, típicamente las de las máquinas que ejecutan los servidores de la aplicación. Así, excepto que el atacante pueda iniciar sesión en el servidor de la aplicación, no va a poder hacer ningún daño aunque consiga la contraseña de acceso de la base de datos.

Autenticación de aplicación

La forma más simple de autenticación consiste en una contraseña secreta que debe ser presentada cuando un usuario se conecta a la aplicación. Desafortunadamente, las contraseñas fácilmente se ven comprometidas (el cifrado es la base de los esquemas de autenticación más robustos). Muchas aplicaciones usan la **autenticación de dos factores o de doble factor**, donde existen dos factores independientes, que no tienen vulnerabilidades comunes, usados para identificar al usuario. Las contraseñas se usan como primer factor en muchos de los esquemas de autenticación de dos factores. Tarjetas inteligentes o otros dispositivos cifrados conectados a través de interfaces USB se usan ampliamente como segundos factores. También se usan como segundos factores los dispositivos de contraseña de un sólo uso (generan un nuevo número pseudoaleatorio cada minuto) o enviar un SMS con una contraseña de un sólo uso al teléfono del usuario.

La autenticación de dos factores puede ser vulnerable a ataques **man-in-the-middle** (hombre en el medio). En estos ataques el usuario intenta conectarse a la aplicación a través de un sitio web falso que acepta contraseñas del usuario y las usa inmediatamente para iniciar sesión en el sitio web original. El protocolo HTTP se usa para autenticar el sitio web para el usuario, además de que cifra los datos, previniendo estos ataques.

Cuando los usuarios acceden a múltiples sitios web, a menudo es incómodo para el usuario tener que registrarse en cada web de forma separada, con diferentes contraseñas para cada sitio. Existe un sistema que permite al usuario autenticarse en un único servicio centralizado de autenticación, y otros sitios web y aplicaciones pueden autenticar al usuario a través de este servicio centralizado.

Un **sistema de inicio de sesión único** permite al usuario autenticarse una vez, y las diferentes aplicaciones autenticarán al usuario a través del servicio de autenticación.

El Lenguaje de Marcado para Confirmaciones de Seguridad (**Security Assertion Markup Language**, SAML) es un estándar para intercambiar información de autenticación y autorización entre diferentes dominios de seguridad, para proveer un sistema de inicio de sesión único entre diferentes organizaciones.

Autorización a nivel de aplicación

La autorización SQL juega un rol muy limitado en el manejo de autorizaciones de usuarios:

- **Falta de información del usuario final.** Los usuarios finales típicamente no tienen identificadores de usuario individuales en la base de datos y de hecho puede haber un único identificador de usuario en la base de datos correspondiente a todos los usuarios de un servidor de aplicación. Así, la especificación de autorización de SQL no puede ser usada sobre este escenario.
- **Falta de autorización detallada.** La autorización debe ser a nivel de tuplas individuales en algunos casos. Este tipo de autorización no es posible en SQL, donde se permite autorización a nivel de relación/tabla o vista, o a nivel de atributos específicos dentro de las mismas.

La tarea de la autorización hoy en día típicamente es realizada de forma completa por la aplicación. A nivel de aplicación, los usuarios son autorizados para acceder a interfaces concretas, y pueden tener solamente permitido el visionado o actualización de ciertos objetos de datos.

Realizar la autorización a nivel de aplicación tiene problemas también:

- El código para comprobar la autorización se mezcla con el resto de código de la aplicación.
- Implementar autorización a través de código de aplicación hace difícil asegurarse de la ausencia de lagunas, ya que por un descuido, un programa de aplicación podría no comprobar la autorización, permitiendo así el acceso a datos confidenciales de usuarios no autorizados.

Algunos sistemas de BBDD proveen mecanismos para una autorización más detallada, como el VPD (Virtual Private Database), que provee autorización a nivel de tuplas específicas.

Trazas de autoría

Las **trazas de autoría** son un registro de todos los cambios (inserciones, borrados y actualizaciones) en los datos de la aplicación, junto con información como el usuario que realizó el cambio y cuándo lo ha realizado. Si la seguridad de la aplicación es violada (o simplemente se ha realizado alguna actualización de forma errónea), una traza de autoría puede ayudar a saber qué ha ocurrido o quién ha realizado esas acciones y a arreglar el daño causado por la brecha de seguridad o por la actualización errónea.

También se usan para detectar brechas de seguridad cuando una cuenta de usuario es comprometida y accedida por un intruso.

Es posible crear trazas de autoría a nivel de BBDD definiendo los disparadores apropiados en las actualizaciones de las relaciones. Sin embargo, muchos sistemas de BBDD proveen mecanismos incorporados para crear traza de autoría. Este tipo de trazas de autoría son usualmente

insuficientes para las aplicaciones, ya que no pueden rastrear el usuario final de las mismas. Además, las actualizaciones son grabadas a bajo nivel, en términos de actualización de tuplas de una relación, cuando lo conveniente es saber las acciones de alto nivel relacionadas con esas actualizaciones de bajo nivel, así como cuándo y quién (dirección IP) ha realizado dicha acción.

Hay que proteger a las trazas de autoría en sí de ser modificadas o borradas por usuarios que violen la seguridad de la aplicación. Una posible solución a esto es copiar las trazas de autoría en una máquina diferente, a la cuál el intruso no tendrá acceso. Se realizaría la copia tan pronto como se genere la traza.

Privacidad

Las aplicaciones que tienen acceso a datos privados deben ser diseñadas cuidadosamente. La privacidad de los usuarios está regulada mediante leyes que los diseñadores de aplicaciones deben cumplir.

7. Cifrado y sus aplicaciones

El cifrado se refiere al proceso mediante el cual se transforman datos a un formato ilegible, excepto que se realice el proceso contrario de descifrado. Los algoritmos de cifrado usan una clave de cifrado para realizar el mismo, y requieren de una clave de descifrado para obtener el mensaje original.

El cifrado es usado ampliamente hoy en día para proteger datos en tránsito entre varias aplicaciones como puede ser la transferencia de datos a través de Internet, o en las redes móviles. También se usa para llevar a cabo otras tareas, como la autenticación.

En el contexto de las bases de datos, el cifrado se usa para almacenar datos de una forma más segura, de forma que, aunque los datos fuesen obtenidos por un usuario no autorizado, estes no podrían ser accedidos sin la clave de descifrado.

Para reducir la posibilidad de que información sensible sea adquirida por criminales, muchos países requieren mediante leyes que todas las bases de datos que almacenen este tipo de información lo hagan mediante cifrado.

Técnicas de cifrado

Hay un amplio número de técnicas de cifrado de datos. Las técnicas simples no proveen la seguridad adecuada a las aplicaciones, ya que es más fácil para un usuario no autorizado de obtener la clave de descifrado.

Una buena técnica de cifrado tiene las siguientes propiedades:

- Es relativamente simple para usuarios autorizados cifrar y descifrar datos.
- No depende del secreto del algoritmo, sino más bien en un parámetro del algoritmo llamado clave de cifrado, que se usa para cifrar los datos. En una técnica de cifrado de **clave simétrica**, la clave de cifrado se usa también como clave de descifrado. Por otro lado, en las técnicas de cifrado de **clave pública** (o clave asimétrica), hay dos claves

diferentes: la clave pública y la clave privada, usadas para cifrar y descifrar datos, respectivamente.

- Su clave de descifrado es extremadamente difícil de determinar por un intruso, aunque tenga acceso a los datos encriptados. En el caso del cifrado de clave asimétrico, es extremadamente difícil de determinar la clave privada aún sabiendo la clave pública.

El **Advanced Encryption Standard (AES)** es un algoritmo de cifrado de clave simétrica que fue adoptado como cifrado estándar por el gobierno de EEUU en el año 2000, y que ahora es ampliamente usado. El estándar está basado en el algoritmo Rijndael.

Para que funcione cualquier esquema de cifrado de clave simétrica, los usuarios autorizados recibirán la clave de encriptado mediante un medio seguro. Este requisito es la mayor debilidad, ya que el esquema no es más seguro que la seguridad del mecanismo por el que es transmitida la clave.

El **cifrado de clave pública** es una alternativa de esquema que evita los problemas del cifrado de clave simétrica. Está basado en dos claves: la pública y la privada. Cada usuario U_i tiene una clave pública E_i y una clave privada D_i . Todas las claves públicas son publicadas, pudiendo ser vistas por cualquiera. Cada clave privada es solo conocida por el usuario al que pertenece. Si el usuario U_i quiere almacenar datos cifrados, U_i los cifra usando E_i . El descifrado requiere D_i .

Para que el cifrado de clave pública funcione debe tener un esquema de cifrado en el cual sea extremadamente difícil deducir la clave privada dando la clave pública. Ese esquema existe y está basado en estas condiciones:

- Hay un algoritmo eficiente para comprobar si un número es primo o no.
- No se conoce ningún algoritmo eficiente para encontrar los factores primos de un número.

A pesar de que el cifrado de clave pública mediante este esquema es seguro, es computacionalmente muy costoso. Un esquema híbrido ampliamente usado para comunicaciones seguras es el siguiente: una clave de cifrado simétrica es aleatoriamente generada e intercambiada de forma segura usando el esquema de cifrado de clave pública, y el cifrado de clave simétrica con esa clave se utiliza en los datos transmitidos posteriormente.

El cifrado de valores pequeños, como nombres o identificadores, se hace complicado por la posibilidad de **ataques de diccionario**. Estos ataques pueden ser disuadidos añadiendo bits extras aleatorios al final del valor antes del cifrado (y borrándolos después del descifrado).

Cifrado soportado en las bases de datos

En el contexto de las bases de datos, el cifrado puede ser realizado a diferentes niveles. En el nivel más bajo, los bloques de disco que contienen datos de la BBDD pueden ser cifrados usando la clave disponible por el software del sistema de la BBDD. Cuando el bloque es recuperado del disco, se descifra antes de usarlo. El cifrado a nivel de bloque de disco protege la BBDD contra atacantes que pueden acceder al contenido del disco pero no a la clave de cifrado.

En el siguiente nivel, atributos específicos de una relación pueden ser almacenados de forma cifrada. En este caso, cada atributo debería tener una clave de cifrado diferente. Muchas BBDD soportan cifrado a nivel de atributos específicos así como a nivel de relaciones enteras, o todas las relaciones de la BBDD directamente. El cifrado de atributos específicos permite cifrar solamente aquellos que contengan información sensible, minimizando así la sobrecarga del descifrado. Sin embargo, cuando atributos individuales o relaciones son encriptadas, las bases de

datos típicamente no permiten que las claves primarias y externas se puedan encriptar. El cifrado también necesita usar bits extras aleatorios para prevenir los ataques de diccionarios.

Una clave de cifrado maestra única puede ser usada para todos los datos encriptados. Con cifrado a nivel de atributos, pueden ser usadas diferentes claves de cifrado para diferentes atributos. En este caso, las claves de descifrado para diferentes atributos pueden estar almacenadas en un archivo o relación (a menudo referenciada como *wallet*) que a su vez está cifrada usando la clave maestra. La clave maestra debe ser suministrada en el momento de la conexión a la BBDD (de no hacerlo no se tendrá acceso a los datos cifrados). Puede ser almacenada en el programa de la aplicación.

El cifrado a nivel de base de datos tiene la ventaja de que requiere relativamente de poco tiempo y sobrecarga de espacio, y no requiere la modificación de las aplicaciones.

Una alternativa a este cifrado es el de cifrar los datos antes de enviarlos a la base de datos. La aplicación debe cifrar los datos antes de enviarlos y descifrarlos después de ser recuperados.

Cifrado y autenticación

Para la autenticación se puede usar un esquema que involucra un **sistema de desafío-respuesta**. El sistema de la BBDD envía el string (desafío) al usuario. El usuario cifra el string usando una contraseña secreta como clave de cifrado y devuelve el resultado. El sistema de la BBDD puede verificar la autenticación del usuario descifrando el string con la misma contraseña secreta y comprobando el resultado con el string original. Este esquema asegura que no viajen contraseñas a través de la red.

Este esquema añade el beneficio de no almacenar la contraseña secreta en la base de datos. Se usan tarjetas inteligentes para almacenar la clave en un chip incorporado. El sistema operativo de la tarjeta inteligente garantiza que la clave nunca puede ser leída, pero permite que los datos se envíen a la tarjeta para cifrarlos y descifrarlos.

Otra aplicación interesante del cifrado de clave pública es en las **firmas digitales** para verificar la autenticidad de los datos. Las firmas digitales juegan el rol electrónico de las firmas físicas en los documentos. La clave privada es usada para “firmar”, esto es, encriptar, los datos, y los datos firmados pueden hacerse públicos. Cualquiera puede verificar la firma descifrando los datos usando la clave pública, pero nadie puede generar datos firmados sin tener la clave privada. Así, podemos autenticar datos. Además, las firmas digitales también sirven para garantizar el no repudio. Esto es, en el caso de que la persona que creó los datos después reclame que él no los ha creado, se puede comprobar que esa persona ha tenido que crearlos (excepto que su clave privada se haya filtrado).

La autenticación es en general un proceso en dos direcciones, donde cada una de las entidades que interactúan se autentican la una a la otra. Tal autenticación por pares es necesitada incluso cuando un cliente contacta con un sitio web, para prevenir sitios web maliciosos enmascarados como sitios web legales.

Para asegurarse de que un usuario está interactuando con un sitio web auténtico, debe tener la clave pública del sitio web. Esto presenta el problema de cómo el usuario puede obtener la clave pública (si está almacenada en el sitio web, un sitio web malicioso puede suministrar una clave pública diferente y el usuario no tendrá manera de verificar si esa clave es la auténtica). La autenticación puede ser manejada por un sistema de **certificados digitales**, donde las claves públicas son firmadas por una agencia de certificación.

Un sistema de dos niveles supondría una carga excesiva de creación de certificados las autoridades de certificación raíz, por lo que se usa un sistema multinivel en su lugar, con una o más autoridades de certificación raíz y un árbol de autoridades de certificación debajo de cada raíz. Cada autoridad tiene un certificado digital emitido por su padre.

Un certificado digital emitido por una autoridad de certificación A consiste en una clave pública K_A y un texto cifrado E que puede ser decodificado usando la clave pública K_A . El texto cifrado contiene el nombre de la parte a que se emitió el certificado y su clave pública K_C . En caso de que la autoridad de certificación A no sea raíz, el texto cifrado contiene también el certificado digital emitido a A por su autoridad de certificación padre. Este certificado autentica la propia clave K_A . Para verificar un certificado, el texto cifrado E es descifrado usando la clave pública K_A para recuperar el nombre de la parte (el nombre de la organización dueña del sitio web). Si A no es raíz, la clave pública K_A es verificada recursivamente usando el certificado digital contenido en E . Verificar el certificado establece la cadena a través la cual un sitio web particular fue autenticado, y provee el nombre y la clave pública autenticada del mismo.

Los certificados digitales son ampliamente usados para autenticar sitios web a los usuarios, para prevenir sitios web maliciosos enmascarados como otros sitios web. En el protocolo HTTPS, los sitios web proveen su certificado digital al navegador, quien se lo muestra al usuario. Si el usuario acepta el certificado, el navegador usa la clave pública proporcionada para cifrar datos. Para reducir los costes del cifrado de clave pública/privada, HTTPS actualmente crea una clave simétrica privada de un sólo uso después de la autenticación, y la usa para cifrar datos por el resto de la sesión.

Los certificados digitales también son usados para autenticar usuarios. El usuario debe enviar un certificado digital que contenga su clave pública al sitio web, el cual verifica que el certificado fue firmado por una autoridad de confianza. La clave pública del usuario puede ser usada en un sistema de desafío-respuesta para asegurarse de que el usuario posee la correspondiente clave privada, autenticando así al usuario.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Diseño de Software

19-04-2021

Clase 8

Temas trabajados:

Tema 4 parte 1.

Índice

1. Introducción al tema	3
-------------------------------	---

1. Introducción al tema

Mosquera da una breve explicación de la primera parte del tema 4, un tema más introductorio que los anteriores temas y que comienza hablando sobre la gestión de memoria en una base de datos, como se guardan los datos y que tipos de almacenamiento podremos encontrar, un repaso a la jerarquía de memoria.

La segunda parte de la lectura habla sobre la organización de los datos en la base de datos, árboles etc., básicamente AED.

Se realizó un Kahoot durante toda la hora de 16 preguntas que os dejo en este enlace:

[KahootBases](#)

Para la semana siguiente propuso leer la segunda parte del tema 4 para realizar otro Kahoot.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Bases de Datos II

26-04-2021



Clase 09

Temas trabajados:

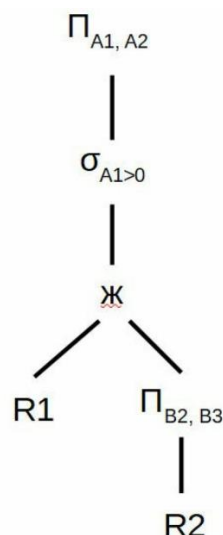
4. Funcionamiento Interno de los SGBD (Parte 2):
Procesamiento y Optimización de Consultas

Índice

1. Kahoot	3
2. Procesamiento de consultas	4
2.1. Descripción general.....	4
2.2. Medidas del coste de una consulta.....	5
2.3. Evaluación de expresiones	6
2.3.1. Materialización.....	6
2.3.2. Encauzamiento.....	6
3. Optimización de consultas	7
3.1. Visión general.....	7
3.2. Transformación de expresiones relacionales.....	7
3.3. Estimación de las estadísticas de los resultados de las expresiones	8
3.4. Elección de los planes de evaluación	8

1. Kahoot

1. **No es un paso del procesamiento de consulta.** Modelado conceptual | Análisis y traducción | Optimización | Evaluación
2. **De estos lenguajes, ¿cuál se emplearía por el optimizador de consultas?** Álgebra Relacional Extendida (ARE) | Structured Query Language (SQL) | Database Access Optimizer (DAO) | Single Optimum Access Protocol (SOAP)
3. **¿Cuál es el coste más importante en una consulta?** Coste del acceso a los datos en disco no volátil | Coste del acceso a los datos en memoria intermedia | Coste de transferencia de datos en disco no volátil | Coste de transferencia de datos en memoria intermedia
4. **Si se pudiera elegir libremente, ¿qué enfoque de conexión de primitivas de evaluación sería preferible?** Encauzamiento | Materialización | Agregación | Reunión Natural
5. **El coste de una evaluación materializada es igual a la suma de los costes de las operaciones involucradas.** Falso
6. **Todas las conexiones de primitivas de evaluación se pueden realizar mediante encauzamiento.** Falso
7. **¿Qué no contiene un plan de ejecución de una consulta?** La sintaxis SQL óptima de la consulta | Las relaciones de la base de datos | Los algoritmos a utilizar en cada operación | La coordinación de operaciones (materialización o encauzamiento)
8. **¿Qué consulta representa el siguiente plan de ejecución (sólo operaciones)?**

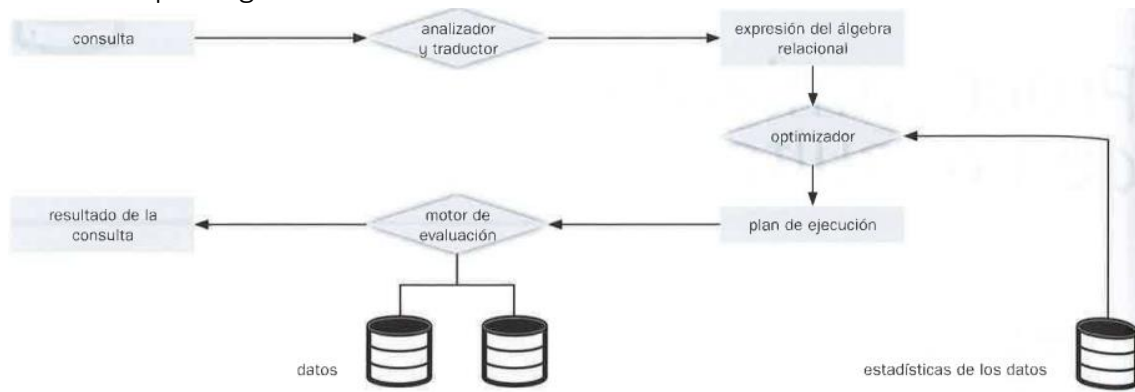


- **SELECT A1, A2; FROM R1, (SELECT B2, B3; FROM R2); WHERE A1 > 0**
- **SELECT B2, B3; FROM R2, (SELECT A1, A2; FROM R1); WHERE A1 > 0**
- **SELECT B2, B3; FROM R2, (SELECT A1, A2; FROM R1; WHERE A1 > 0)**
- **SELECT A1, A2; FROM R1, (SELECT B2, B3; FROM R2; WHERE A1 > 0)**

9. **El optimizador de consultas encuentra siempre el plan de ejecución menos costoso.** Falso
10. **Las consultas se pueden expresar de varias maneras diferentes, con costes de evaluación distintos.** Verdadero
11. **¿Qué tiene que coincidir para que dos expresiones del álgebra relacional se consideren equivalentes?** Generar el mismo conjunto de tuplas | Tardar el mismo tiempo | Generar las tuplas en el mismo orden | Estar formadas por las mismas primitivas de evaluación
12. **Las estimaciones de estadísticas, usadas en la optimización de consultas, no son muy precisas.** Verdadero
13. **Las optimizaciones usan reglas generales que reducen el coste de optimización (a riesgo de no encontrar el plan óptimo).** Verdadero
14. **¿Cómo puede un programador mejorar el coste de una consulta?** Adelantando todo lo posible las operaciones de proyección y selección | Retrasando todo lo posible las operaciones de proyección y selección | Adelantando todo lo posible las operaciones de reunión | No se puede hacer nada para mejorar el coste de una consulta

2. Procesamiento de consultas

2.1. Descripción general



Aquí se muestran los pasos involucrados en el procesamiento de una consulta. Los básicos son:

- Análisis y traducción.
- Optimización.
- Evaluación.

La primera acción que el sistema tiene que realizar para procesar una consulta es su traducción a su formato interno, es decir, álgebra relacional (más útil que el SQL). Este proceso es semejante al que realiza el analizador de un compilador.

Cada consulta SQL puede traducirse a diversas expresiones del álgebra relacional.

Para especificar completamente cómo evaluar una consulta, no basta con proporcionar la expresión del álgebra relacional, además hay que anotar en ella las instrucciones que especifiquen

cómo evaluar cada operación. Las operaciones del álgebra relacional anotadas con instrucciones sobre su evaluación se llaman **primitivas de evaluación**. La secuencia de operaciones primitivas que se pueden utilizar en la evaluación de una consulta establece un **plan de ejecución de la consulta** o un **plan de evaluación de la consulta**. El **motor de ejecución de consultas** escoge un plan de evaluación, lo ejecuta y devuelve su respuesta a la consulta. Cada plan de evaluación puede tener un coste distinto. Es responsabilidad del sistema que este coste sea mínimo. Para optimizar una consulta, el optimizador de consultas debe conocer el coste aproximado de cada operación.

Cauce: agrupación de varias operaciones donde cada una empieza trabajando sobre sus tuplas de entrada del mismo modo que si fuesen generadas por otra operación.

2.2. Medidas del coste de una consulta

Existen muchos posibles planes de evaluación de una consulta y es importante compararlos dependiendo de su coste. Para ello hay que estimar el coste de las operaciones individuales y combinarlas para obtener el coste de un plan de evaluación de una consulta.

El coste de la evaluación de una consulta se puede expresar en términos de diferentes recursos (accesos a disco, tiempo de ejecución de la CPU, etc.).

En grandes BD, el coste de acceso a los datos en disco es normalmente el más importante. Además, la velocidad de la CPU ha aumentado mucho más rápido, entonces, lo más probable es que el tiempo de operaciones del disco domine al de ejecución. Las estimaciones de tiempo de CPU son más difíciles de establecer porque dependen de detalles de bajo nivel del código de ejecución.

Se usará el *nº de transferencia de bloques de disco* y el *nº de búsquedas de disco* para medir el coste de un plan de evaluación de una consulta.

Se puede refinar aún más la estimación del coste distinguiendo entre las lecturas y escrituras de bloques, dado que normalmente se tarda el doble en escribir.

Los costes de todos los algoritmos que se consideran dependen del tamaño de la memoria intermedia en la principal. En el mejor caso, todos los datos se pueden leer en las memorias intermedias. En el peor caso se asume que la memoria intermedia puede contener solo unos pocos bloques de datos. Al presentar estimaciones de coste se asume el peor caso.

El **tiempo de respuesta** para un plan de evaluación de una consulta, suponiendo que no está realizando ninguna otra actividad, tendría en cuenta todos los costes y se podría usar como una medida del coste del plan. Desafortunadamente, el tiempo de respuesta de un plan es muy difícil de estimar sin ejecutar realmente el plan por:

1. El tiempo de respuesta depende del contenido de la memoria intermedia cuando empieza la ejecución de la consulta; no se puede conocer esta información y, aunque se pudiese, resultaría difícil tenerlo en cuenta.
2. En un sistema con varios discos, el tiempo de respuesta depende de la distribución de los accesos a los mismos; resulta difícil estimarlo sin una información detallada de esta distribución.

Curiosamente, un plan puede tener un mejor tiempo de respuesta a costa de un consumo adicional de recursos.

El resultado es que, en lugar de intentar minimizar el tiempo de respuesta, los optimizadores suelen intentar minimizar el **consumo de recursos** total de un plan de consulta.

2.3. Evaluación de expresiones

La manera más evidente de evaluar una expresión es simplemente evaluar una operación a la vez en un orden apropiado. El resultado de cada evaluación se **materializa** en una relación temporal para su inmediata utilización. Un inconveniente de esto es la necesidad de construir relaciones temporales. Un enfoque alternativo es evaluar varias operaciones de manera simultánea en un **cauce** (*pipeline*) con los resultados de una operación que se pasan a la siguiente, sin almacenar relaciones temporales.

2.3.1. Materialización

Con una representación gráfica de la expresión en un **árbol de operadores** es más fácil entender cómo evaluar una expresión. Se empieza por las operaciones de la expresión de nivel más bajo. Se ejecutan las operaciones y se almacenan los resultados en relaciones temporales. Luego estas se utilizan para ejecutar las operaciones del siguiente nivel, cuyas entradas son ahora o relaciones temporales o relaciones almacenadas en la BD.

Repitiendo el proceso se calcularía finalmente la operación en la raíz del árbol, obteniendo el resultado final de la expresión.

Una evaluación como la descrita se llama **evaluación materializada**. Su coste no es simplemente la suma de los costes de las operaciones involucradas. Para calcularlo hay que añadir los costes de todas las operaciones, incluyen el coste de escribir los resultados intermedios en el disco.

La **memoria intermedia doble** permite que el algoritmo se ejecute más rápidamente mediante la ejecución en paralelo de acciones en la CPU con acciones E/S. El número de búsquedas se puede reducir asignando bloques adicionales en la memoria intermedia de salida y escribiendo varios bloques a la vez.

2.3.2. Encauzamiento

Se puede mejorar la eficiencia de la evaluación de consultas mediante la reducción del número de archivos temporales que se producen. Esto se lleva a cabo mediante la combinación de varias operaciones relacionales en un **encauzamiento** de operaciones, donde se pasan los resultados de una operación a la siguiente. Esta evaluación se llama **evaluación encauzada**.

La creación de un encauzamiento de operaciones tiene 2 ventajas:

1. Elimina el coste de leer y escribir relaciones temporales, reduciendo el coste de la evaluación de consultas.
2. Puede empezar a generar resultados de la consulta rápidamente, si el operador raíz del plan de evaluación se combina en un encauzamiento con las entradas. Esto puede resultar muy útil si los resultados se muestran al usuario según se generan ya que, en caso contrario, puede haber un gran retardo antes de que el usuario vea algún resultado.

3. Optimización de consultas

3.1. Visión general

La **optimización de consultas** es el proceso de selección del plan de evaluación de las consultas más eficientes de entre las muchas estrategias. Se espera que el sistema cree un plan de evaluación que minimice el coste de la evaluación de consultas.

Un aspecto de esta optimización tiene lugar en el nivel del álgebra relacional, donde el sistema intenta hallar una expresión que sea equivalente a la expresión dada, pero de ejecución más eficiente. Otro aspecto es la elección de una estrategia para el procesamiento de la consulta.

La diferencia en coste entre una estrategia buena y una mala suele ser sustancial. Por tanto, merece la pena que el sistema invierta una cantidad importante de tiempo en la selección de una buena para la consulta, aunque solo se ejecute una vez.

Dada una expresión del álgebra relacional, es labor del optimizador de consultas diseñar un plan de evaluación que calcule el mismo resultado que la expresión dada, y de la forma menos costosa a la hora de generar el resultado.

Para encontrar el plan de evaluación de consultas menos costoso el optimizador necesita generar planes alternativos que produzcan el mismo resultado que la expresión dada y escoger el de menor coste. La generación de estos planes consta de 3 etapas:

1. La generación de expresiones que sean equivalentes lógicamente a la expresión dada.
2. La anotación de las expresiones alternativas resultantes para generar planes distintos de ejecución.
3. La estimación del coste de cada plan de evaluación, y la elección del que se estime que tiene menor coste.

Las etapas (1), (2) y (3) se encuentran entrelazadas en el optimizador de consultas; algunas expresiones se generan y se anotan para dar lugar a planes de evaluación, luego se generan y se anotan otras expresiones, etc. Según se generan planes de evaluación, se estiman sus costes usando información estadística sobre las relaciones, como los tamaños de las relaciones y la profundidad de los índices.

Para implementar (1) el optimizador de consultas debe generar expresiones equivalentes a la dada. Esto se lleva a cabo mediante las *reglas de equivalencia*, que especifican el modo de transformar una expresión en otra lógicamente equivalente.

Se puede escoger un plan de evaluación de consultas basado en el coste estimado de los planes. Dado que el coste es una estimación, el plan seleccionado no es necesariamente el de menor coste; no obstante, siempre y cuando las estimaciones sean buenas, es probable que el plan sea el de menor coste, o no represente mucho más coste.

Finalmente, las vistas materializadas ayudan a acelerar el procesamiento de ciertas consultas.

3.2. Transformación de expresiones relacionales

Las consultas se pueden expresar de varias maneras distintas, con costes de evaluación distintos.

Se dice que 2 expresiones del álgebra relacional son **equivalentes** si, en cada ejemplar legal de la BD, las 2 generan el mismo conjunto de tuplas.

En SQL las entradas y las salidas con multiconjuntos de tuplas, y se usa la versión para multiconjuntos del álgebra relacional para evaluar las consultas de SQL. Se dice que 2 expresiones

de la versión *para multiconjuntos* del álgebra relacional son equivalentes si en cada BD legal las 2 expresiones generan el mismo multiconjunto de tuplas.

3.3. Estimación de las estadísticas de los resultados de las expresiones

El coste de cada operación depende del tamaño y de otras estadísticas de sus valores de entrada.

Las estimaciones no son muy precisas, ya que se basan en suposiciones que pueden no cumplirse exactamente. El plan de evaluación de consultas que tenga el coste estimado de ejecución más reducido puede, por tanto, no tener el coste real de ejecución más bajo. Sin embargo, la experiencia real demuestra que, aunque las estimaciones no sean muy precisas, los planes con los costes estimados más reducidos tienen costes de ejecución reales que, o bien son los más reducidos, o bien se hallan cercanos al menor coste real de ejecución.

3.4. Elección de los planes de evaluación

La generación de expresiones es solo una parte del proceso de optimización de consultas, ya que cada operación de la expresión puede implementarse con algoritmos diferentes. Un plan de evaluación define exactamente el algoritmo que se usará para cada operación y el modo en que se coordinará la ejecución de las operaciones.

Dado un plan de evaluación, se puede estimar su coste empleando las estadísticas estimadas mediante técnicas junto con las estimaciones de costes de varios algoritmos y métodos de evaluación vistos anteriormente.

Un **optimizador basado en el coste** explora el espacio de todos los planes de evaluación de consultas equivalentes a la consulta dada y selecciona la que tenga el menor coste estimado. La optimización basada en coste con reglas de equivalencia arbitrarias es muy complicada.

Explorar el espacio de todos los planes posibles puede ser muy costoso para consultas complejas. La mayoría de los optimizadores incluyen heurísticas que reducen el coste de la optimización de consultas, con el riesgo de no encontrar el plan óptimo.



Escola
Técnica
Superior
de Enxeñaría



Grado en Enxeñaría Informática

2º curso – 2º cuadrimestre

Bases de Datos II

03-05-2021



Clase 10

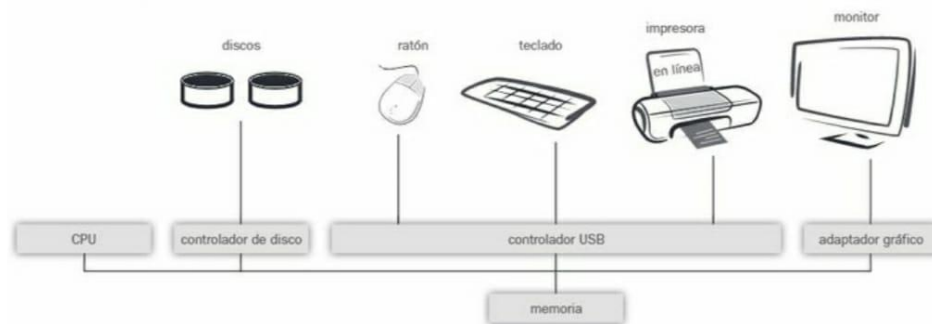
Temas trabajados: Tema 5. Arquitecturas dos
SGBD: Bases de Datos Paralelas e Distribuidas.
Kahoot

Índice

1. Kahoot sen respostas	3
2. Kahoot con respostas	5

1. Kahoot sen respostas

1. A que tipo de Sistema responde a figura?



- Sistemas Procesador de transacciones
- Sistema Acceso a Datos
- Sistema Centralizado
- Sistema Cliente-Servidor

2. Os sistemas centralizados poden ser multiusuario?

- Verdadeiro
- Falso

3. Que sistemas de base de datos carecen de control de concurrencia?

- Cliente-Servidor
- Centralizados multiusuario
- Sistemas distribuídos
- Centralizados monousuario

4. Nos sistemas centralizados multiusuario o paralelismo establécese habitualmente a nivel de ...

- Bloques de datos (1 bloque de datos a cada procesador)
- Rexistros (1 tupla a cada procesador)
- Transacciones (1 transacción a cada procesador)
- Instrucciones (1 instrucción SQL a cada procesador)

5. A que tipo de Sistema responde a figura?



- Sistemas Procesador de transaccións
- Sistema Acceso a Datos
- Sistema Cliente-Servidor
- Sistema Centralizado

6. A conexión mediante ODBC (e calquera das súas versións particulares) é propia dunha arquitectura...

- Cliente-Servidor
- Centralizada Monousuario
- Desconectada
- Multiusuario

7. Que arquitectura é a máis usada amplamente nos servidores?

- Servidores de rexistros
- Servidores de datos
- Servidores de bloques
- Servidores de transaccións

8. Que mellora o paralelismo de consultas?

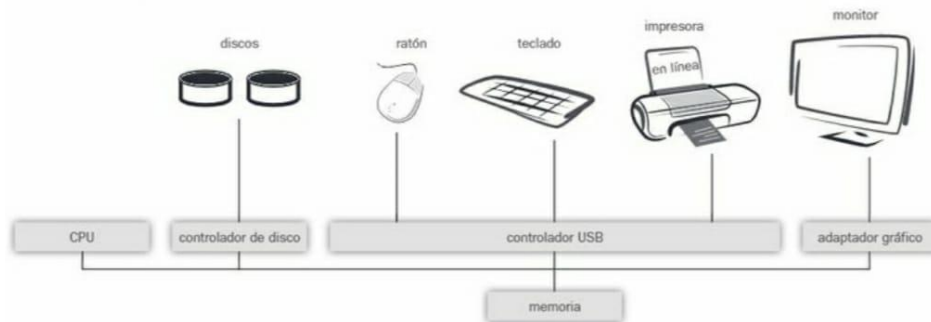
- A fiabilidade do sistema
- Os modelos de datos
- A velocidade de resposta
- O número de datos necesarios

9. Cando os diferentes sitios dunha arquitectura distribuída empregan idéntico software de xestión falase...

- Bases de datos distribuídas homoxéneas
- Bases de datos distribuídas heteroxéneas
- Bases de datos paralelas homoxéneas
- Bases de datos paralelas heteroxéneas

2. Kahoot con respuestas

1. A que tipo de Sistema responde a figura?



- Sistemas Procesador de Transacciones
- Sistema Acceso a Datos
- **Sistema Centralizado**
- Sistema Cliente-Servidor

As arquitecturas máis básicas son as centralizadas, cando se desenvolve o modelo relacional (sobre os anos 70), estes eran os sistemas predominantes. Nacen nunha serie de entornos que tiñan unha necesidade de procesar datos, o entorno bancario era o máis representativo. As liñas de interconexión entre estes sistemas eran dedicadas, e a eles accedíase a través de terminais, sen interfaces gráficas. Estes permiten conectar todo a través dun único bus de conexión, o que permite xestionar a seguridade dunha forma moito máis sinxela que nos sistemas distribuídos, ó necesitarse unha conexión física a este bus para acceder ós datos. O poder implementar bases de datos relacionais neste tipo de sistemas tamén condicionou o desenvolvemento das bases de datos, xa que se complica a adopción de novos modelos.

2. Os sistemas centralizados poden ser multiusuario?

- **Verdadeiro**
- Falso

Non se debe confundir centralizado con monousuario. Un exemplo de sistemas centralizados monousuario poden ser as típicas bases de datos de paquetes ofimáticos.

3. Que sistemas de base de datos carecen de control de concurrencia?

- Cliente-Servidor
- Centralizados multiusuario
- Sistemas distribuídos
- **Centralizados monousuarios**

Por carecer desta característica, este tipo de sistemas son moito máis sinxelos de implementar. Non o necesitan porque co sistema só interacciona un usuario, que espera a que se complete a transacción antes de lanzar unha nova.

4. Nos sistemas centralizados multiusuario o paralelismo establécese habitualmente a nivel de ...

- Bloques de datos (1 bloque de datos a cada procesador)
- Rexistros (1 tupla a cada procesador)
- **Transaccións (1 transacción a cada procesador)**
- Instrucións (1 instrución SQL a cada procesador)

Se temos un sistema centralizado multiusuario, é dicir, temos varios procesadores dentro da nosa conexión, podemos aproveitar este paralelismo para lanzar unha transacción a cada procesador e deste xeito acelerar a operación, mais cando nestes sistemas centralizados temos esta posibilidade temos que ter coidado e utilizar sistemas de control de concurrencia. Foi no desenvolvemento das arquitecturas centralizadas multiusuario cando este tipo de sistemas de control se comezaron a desenvolver. Cando hai varias operacións de lectura non hai problema, mais as operacións de escritura si que poden traer problemas.

Os sistemas centralizados tiveron o seu auxe nos anos 80, mais ó chegar á década dos 90 comeza a desenvolverse internet, e progresivamente vanse mellorando as conexións de maneira que estes sistemas pasan a segundo plano (non desaparecen, xa que existen empresas e institucións que utilizan este tipo de sistemas convivindo con sistemas abertos e distribuídos, xa que os riscos de accesos non controlados neste último tipo de arquitectura é maior. Por exemplo, un banco, para realizar un acceso ós datos coa finalidade de facer unha análise interna, podería optar por realízalos sobre un sistema centralizado, en lugar de optar pola rede pública)

5. A que tipo de Sistema responde a figura?



- Sistemas Procesador de transaccións
- Sistema Acceso a Datos
- **Sistema Cliente-Servidor**
- Sistema Centralizado

Esta arquitectura vén substituír á arquitectura centralizada. Esta é a arquitectura dominante na actualidade, xa que permite un nivel de interconexión moito máis alto tendo en conta que os niveis de seguridade son máis baixos en comparación coa arquitectura centralizada.

6. A conexión mediante ODBC (e calquera das súas versións particulares) é propia dunha arquitectura...

- **Cliente-Servidor**
- Centralizado Monousuario
- Desconectada

- Multiusuario

Normas ODBC: normas de interconexión.

Norma de conexión que permite a un cliente que quere realizar unha petición de información: como se establece a conexión ó servidor, como se lle envía ó servidor a necesidade de información e como se recibe do servidor a información cos resultados. Íllase a base de datos do lado do servidor e dende o lado do cliente hai diferentes posibilidades: diferentes navegadores, apps... Compartindo todas as mesmas normas de conexión.

7. Que arquitectura é a máis usada amplamente nos servidores?

- Servidores de rexistros
- Servidores de datos
- Servidores de bloques
- **Servidores de transaccións**

Unha vez se realiza a conexión ó servidor, indícaselle a transacción a resolver e o servidor devólvenos o resultado.

Servidores de datos (moito menos utilizados): Non se lle indica a necesidade de información que se desexa, se non que dos datos que posúe almacenados, indícaselle cales interesan. Será o cliente o encargado de procesar estes datos. A diferenza é que en vez de procesar o servidor a transacción, este só devolve os datos e é o programa o que os procesa e fai operacións sobre eles.

8. Que mellora o paralelismo de consultas?

- A fiabilidade do sistema
- Os modelos de datos
- **A velocidade de resposta**
- O número de datos necesarios

Tense en conta o *paralelismo de gran groso*, collendo cada tarefa que hai que realizar e dándolla a cada procesador do ordenador, tendo en conta as limitación que impón o sistema de concurrencia, os resultados deses procesos en paralelo van ser executados.

O illamento de instantáneas vaise impoñendo como sistema de concurrencia, dándolle así ós distintos procesadores os datos e variables sobre os que vai operar. Hai que recordar que o illamento de instantáneas, no momento final, hai que ver se o traballo que fixo o procesador nos vale porque supera os controis de concurrencia, ou pola contra hai que retroceder.

9. Cando os diferentes sitios dunha arquitectura distribuída empregan idéntico software de xestión falase...

- **Bases de datos distribuídas homoxéneas**
- Bases de datos distribuídas heteroxéneas
- Bases de datos paralelas homoxéneas
- Bases de datos paralelas heteroxéneas

As bases de datos distribuídas non están en un só equipo físico, se non que aproveitando a interconexión as bases de datos están en diferentes elementos. Nas bases de datos distribuídas homoxéneas, de ese modelo de datos, unha parte deles están nuns ordenadores mentres que outra parte esta en outra. Non é unha división trivial, porque se resulta que facemos unha división na que parte dos datos están en distintos ordenadores, se unha consulta implica datos de varios ordenadores, ademais de executar a debida sentença SQL, tamén se deben reunir os datos e polo tanto o tempo de traspaso de datos vai ser de gran importancia e terá moito impacto no tempo de execución. (Existe unha xerarquía ó realizar transaccións neste tipo de sistemas, xa que son as máquinas de nivel máis alto da xerarquía as que lle indican ás de nivel menor que operacións teñen que realizar e estas á súa vez, poden realizar operacións sobre outras... Esta distribución faise de forma que a conxestión nas comunicacións sexa mínima, necesitando ter outros factores en conta. A isto hai que sumarlle que a base de datos é distribuída e tamén hai que controlar a concurrencia... A complexidade é tal que sae do que pode acaparar a materia).

Por outro lado temos as bases de datos distribuídas heteroxéneas (di Mosquera que son máis interesantes que as homoxéneas). Ó contrario que sucede coas homoxéneas, nas bases de datos heteroxéneas existe unha maior autonomía individual por parte de cada máquina e non se produce esa cooperación que si ten lugar nas homoxéneas. As heteroxéneas teñen un funcionamento inverso ás homoxéneas. As bases de datos dos sitios locais existen a priori e son distintas, este tipo de problemas denomínase como: “*problemas de federación de bases de datos*”. O problema destas bases de datos é que se poden atopar distintos sistemas operativos, xestores de bases diferentes... Ademais pode que non se queira compartir toda a información, xa que unha organización pode decidir unicamente compartir unha parte da base de datos (para compartir só unha parte da base de datos pódese facer uso de vistas). Mais como se combinan modelos de datos que son heteroxéneos para crear un modelo de datos nunha capa superior que os reúna? Hai moitos problemas, xa que hai que integrar nun único modelo global distintas partes de distintas bases de datos, e tamén hai a necesidade de poder executar instrucións sobre ese modelo global. Mais isto tamén ten a vantaxe de que se temos organizacións cunha traxectoria consolidada en canto á base de datos, non hai que cambiar a base de datos completa, se non que só hai que cambiar o modelo local e deste xeito, o modelo global xa vai recoller os modelos locais como a súa base para realizar o propio modelo global. Tamén cabe resaltar que neste tipo de sistemas, o control de concurrencia ten unha complexidade moi grande, maior que no caso dos sistemas distribuídos homoxéneos, xa que cada sistema pode tratar a concurrencia de distinta maneira, e polo tanto ten que ser unha capa superior o que faga o control.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Diseño de Software

Si-No-2021

Clase Algo

Temas trabajados:

Temas 6 y 7

Índice

1. Introducción al tema	¡Error! Marcador no definido.
-------------------------------	--------------------------------------

1. Kahoot chingon

